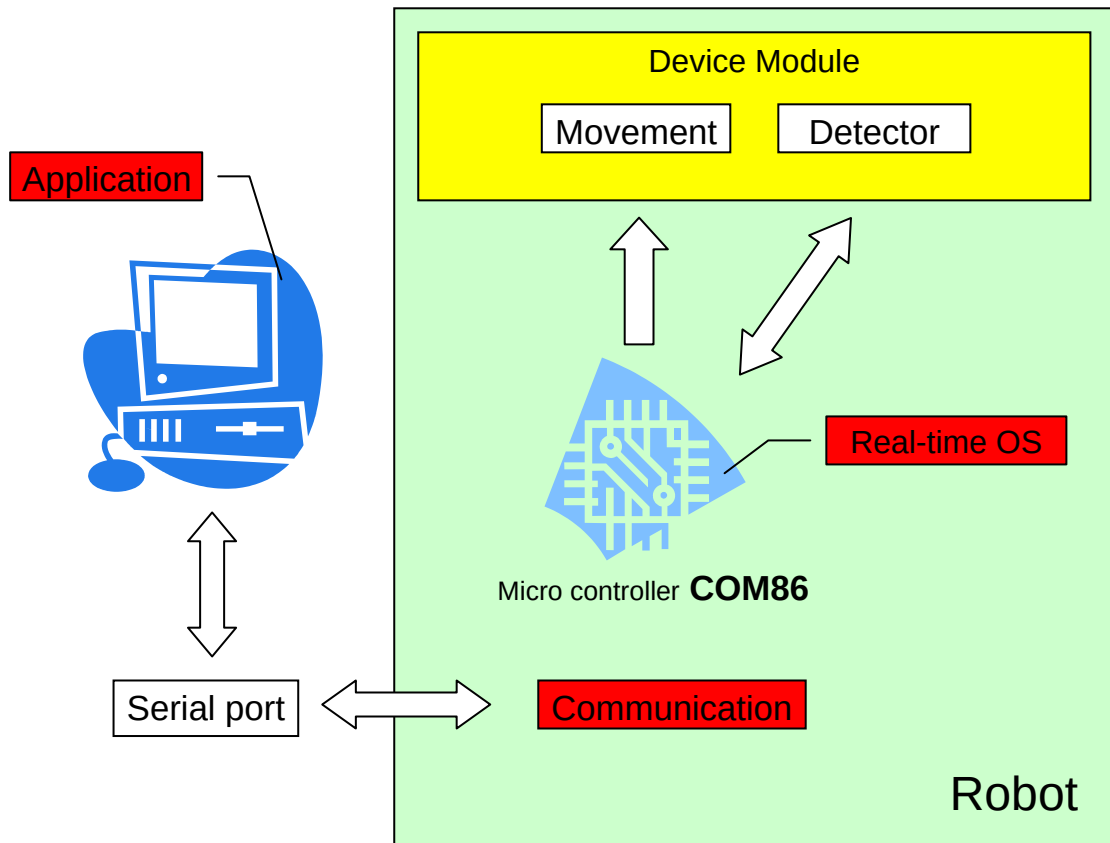


คู่มือสำหรับโปรแกรมเมอร์

การออกแบบและพัฒนาโปรแกรม

โครงการนี้มีจุดประสงค์หลักคือ นำคอม86 มาประยุกต์ใช้ในงานหุ่นยนต์ และออกแบบโปรแกรมที่จะช่วยเหลือผู้ใช้งานในการสร้างหุ่นยนต์รวมถึงออกแบบชุดคำสั่งที่สามารถเรียกใช้ควบคุมอุปกรณ์ได้โดยง่าย รวมถึงโปรแกรมที่จะช่วยแสดงผลการทำงานและควบคุมหุ่นยนต์ผ่านทางคอมพิวเตอร์



รูปที่ 1 คอนเท็กซ์ไดอะแกรม ของโครงการ

จากภาพประกอบด้วยส่วนประกอบ 2 ส่วนด้วยกัน คือ

1. ส่วนของหุ่นยนต์ที่ใช้คอม86เป็นตัวประมวลผลหลักจากบอร์ดคอม86จะต่อเข้ากับ บอร์ดที่ทำ การ ขยายพอร์ตแอดเดสและเชื่อมต่อกับ วงจรของอุปกรณ์ที่ออกแบบเป็น โมดูลมาตรฐานสำหรับเชื่อมต่อกับ ตัว บอร์ด ขยายพอร์ตแอดเดสและใช้ ระบบปฏิบัติการแบบเรียลไทม์ในการจัดการทำงานของคอม86 โดยมีการสื่อสารระหว่างหุ่นยนต์กับคอมพิวเตอร์โดยการส่งข้อมูลผ่านพอร์ตอนุกรม
2. ส่วนของคอมพิวเตอร์ที่จะมีโปรแกรมหลักสำหรับควบคุมการทำงาน ส่วนคอมพิวเตอร์นั้นใช้ สำหรับการเขียน โปรแกรมควบคุมหุ่นยนต์ด้วยภาษาซี ภายใต้การควบคุม โดยโปรแกรมช่วยเหลือสำหรับการ ใช้งานหุ่นยนต์ ที่คณะผู้จัดทำสร้างขึ้นเพื่ออำนวยความสะดวกและจัดการการทำงานของหุ่นยนต์ รวมถึงส่วนของชุดหรัับควบคุมและสั่งงานอุปกรณ์ ที่เป็นชุดคำสั่งสำหรับเรียกใช้งานเพื่อสะดวกและง่าย

ต่อการควบคุมอุปกรณ์บนตัวหุ่นยนต์ รวมถึงโปรแกรมที่ติดต่อส่งผ่านข้อมูลระหว่างหุ่นยนต์กับคอมพิวเตอร์ผ่านทางพอร์ตอนุกรม

จากส่วนประกอบทั้งสองด้านนั้นจำแนกตัวโครงงานออกเป็น 4 ส่วนหลัก ดังนี้

1 ส่วนของการเชื่อมต่ออุปกรณ์กับคอม86

โดยในส่วนนี้ทางคณะผู้จัดทำจะทำการออกแบบวงจรที่จะใช้งานตัวอุปกรณ์เช่น วงจรไคร์พมอเตอร์ วงจรเซนเซอร์ เป็นโมดูลที่มีรูปแบบมาตรฐาน ที่ทำให้ผู้ใช้งานสามารถเรียกใช้งาน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ได้อย่างถูกต้อง

2 ส่วนโปรแกรมควบคุมหลัก

โดยในส่วนนี้เป็นส่วนที่จะช่วยจัดการและดูแลการทำงานของหุ่นยนต์ประกอบด้วยส่วนต่าง ๆ ดังนี้

- ส่วนการจัดการอุปกรณ์ต่าง ๆ ที่นำมาเชื่อมต่อกับหุ่นยนต์
- ส่วนช่วยเหลือผู้ใช้งานในการเขียนโปรแกรมและโหลดโปรแกรมไปยังคอม86
- ส่วนควบคุมและจัดการเมมโมรีการใช้งานโปรแกรมต่าง ๆ บนคอม86
- ส่วนประมวลผลและส่งข้อมูลจากหุ่นยนต์ไปแสดงผลการทำงานที่คอมพิวเตอร์

3 ส่วนของคำสั่งสำหรับผู้ทำให้สามารถควบคุมอุปกรณ์ต่าง ๆ

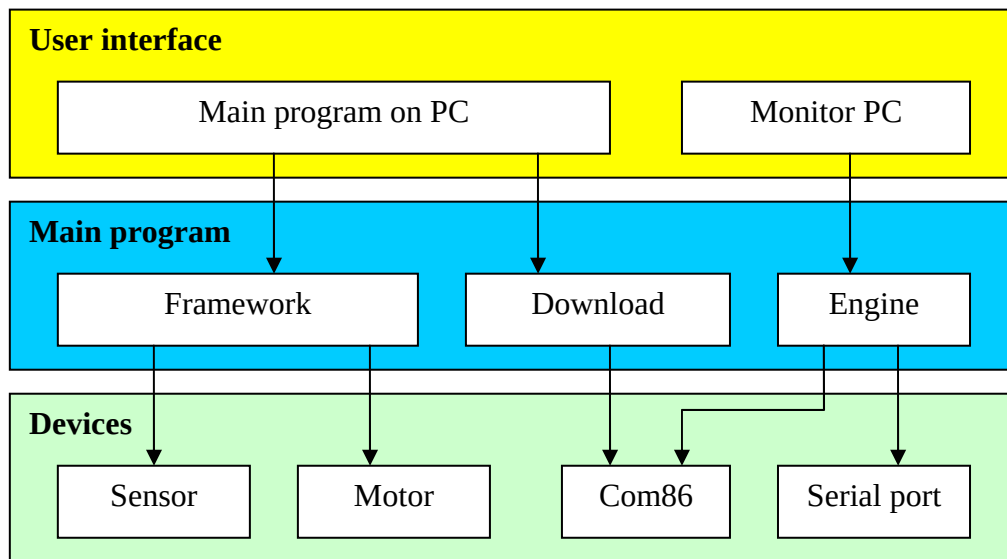
โดยในส่วนนี้ของ ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ นั้นทางคณะผู้จัดทำจะทำการเขียนขึ้น เป็นเหมือนชุดคำสั่งสำหรับตัวอุปกรณ์นั้นๆ โดยเมื่อผู้ใช้งานทำการเพิ่มอุปกรณ์นั้น ๆ เข้ามาทางโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ จะทำการโหลด .h ของอุปกรณ์ตัวนั้นมาลงในโปรแกรมหลัก (ตัวอย่างเช่น Include motor.h) และจะมีคำสั่งต่าง ๆ ที่ให้ผู้ใช้งานสามารถเรียกใช้ควบคุมอุปกรณ์ได้เลยโดยไม่จำเป็นต้องเขียนโปรแกรมติดต่อกับอุปกรณ์โดยตรง

4 ส่วนของโปรแกรมบนคอมพิวเตอร์

(ในขั้นตอนนี้จะทำการพัฒนาโปรแกรมที่ติดต่อกับหุ่นยนต์บนคอมพิวเตอร์ ก่อนแล้วอาจจะทำการเพิ่มเติมไปใช้งานบน Pocket PC หรือ PDA ในภายหลัง) โดยมีส่วนประกอบของโปรแกรมหลัก ๆ ดังนี้

- ส่วนหลักของโปรแกรม เป็นส่วน ติดต่อกับผู้ใช้งานหลักที่จะทำให้ผู้ใช้งาน สามารถเพิ่ม/ลด อุปกรณ์, เขียนโปรแกรม, ส่งข้อมูลไปยังคอม86 รวมถึงส่วนแสดงผลการทำงาน โดยคำสั่งต่าง ๆ ผู้ใช้งานสามารถใช้งานได้ตลอดเวลาเพราะบนหุ่นยนต์นั้นสามารถแก้ไขข้อมูลหรือเปลี่ยนแปลงโปรแกรมในขณะใดก็ได้
- ส่วนแสดงผลการทำงาน เป็นส่วนที่จะแสดงผลการทำงานของหุ่นยนต์ในขณะนั้นโดยจะมีโปรแกรมบนคอม86 จะทำการส่งข้อมูลต่าง ๆ มายังคอมพิวเตอร์ โดยบนคอมพิวเตอร์ จะมีโปรแกรมแสดงผลข้อมูลนั้น ๆ ให้กับผู้ใช้งานทำให้เราสามารถทราบถึงการทำงานและผลของการทำงานของหุ่นยนต์ตลอดเวลา

โครงสร้างทางสถาปัตยกรรมของโครงการ



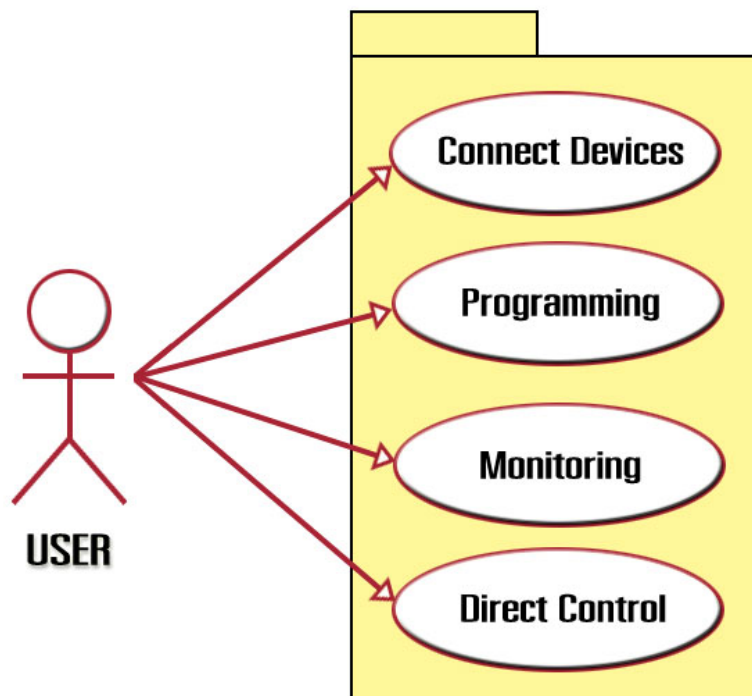
รูปที่ 2 โครงสร้างทางสถาปัตยกรรมของโครงการ

แบ่งโครงสร้างของโปรแกรมออกเป็น 3 ชั้น

- ส่วนแอปพลิเคชันบนคอมพิวเตอร์ที่ติดต่อกับผู้ใช้งาน (User Interface)
- ส่วนโปรแกรมควบคุมการทำงานหลัก (Main Program)
- ส่วนโปรแกรมที่ทำงานบนคอม86 ทำหน้าที่ควบคุมอุปกรณ์ (Devices layer)

โครงสร้างของโปรแกรมจำแนกลำดับชั้นของโครงสร้างโปรแกรมออกเป็น 3 ส่วนหลัก ดังภาพโดยส่วนที่ติดต่อกับผู้ใช้งานนั้นคือส่วนที่แอปพลิเคชันที่ทำงานอยู่บนคอมพิวเตอร์ โดยหน้าที่หลักของส่วนนี้คือช่วยในการสื่อสารระหว่างผู้ใช้งานกับหุ่นยนต์ อีกส่วนหลักก็คือ ส่วนโปรแกรมที่ทำงานติดต่อกับอุปกรณ์ซึ่งเป็นโปรแกรมที่ทำงานในระดับแอดเดสที่เชื่อมอยู่กับอุปกรณ์โดยส่วนนี้จะเหมือนส่วนชุดคำสั่งในระดับล่างที่ผู้ใช้งานสามารถเรียกใช้ได้โดยไม่ต้องทราบถึงการเชื่อมต่อในระดับฮาร์ดแวร์ ส่วนตรงกลางตามโครงสร้างที่วางไว้คือ เป็นส่วนของโปรแกรมควบคุมการทำงาน เป็นส่วนที่คอยจัดการและทำการช่วยเหลือผู้ใช้งาน ในส่วนของทำการเชื่อมต่อคอมพิวเตอร์ กับหุ่นยนต์นั่นเองโดยการทำงานส่วนใหญ่เป็นโปรแกรมที่ทำงานอยู่บนคอมพิวเตอร์

ยูสเคสไดอะแกรม (Use Case Diagram) ของระบบ



รูปที่ 3 ยูสเคสไดอะแกรมของผู้ใช้งาน

1 ผู้ใช้งาน (User)

ความต้องการจากผู้ใช้งานนับเป็นจุดสำคัญของโครงการเนื่องจากโปรแกรมมีจุดประสงค์หลักมาจากการออกแบบโปรแกรมที่จะช่วยเหลือการสร้างหุ่นยนต์ ให้สามารถทำได้โดยง่ายสะดวก เข้าใจได้ง่ายและแก้ปัญหาการขาดทักษะรวมถึงขาดประสบการณ์ในการออกแบบ และทำงานกับอุปกรณ์ต่าง ๆ ของหุ่นยนต์

1.1 ติดตั้งอุปกรณ์กับบอร์ด คอม86

ผู้ใช้งานสามารถต่อโมดูลของอุปกรณ์ที่สนับสนุนการทำงานของฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ กับชุดพัฒนา คอม86 โดย ผู้ใช้งานจะต่อโมดูลของอุปกรณ์ กับบอร์ดขยายพอร์ต แอแดปเตอร์ ที่ขยายพอร์ตจากชุดพัฒนา คอม86 ที่คณะผู้จัดทำกำหนดไว้ให้ และ ผู้ใช้งานจะสามารถปรับแต่งบนแอปพลิเคชัน ด้วยว่าได้ต่ออุปกรณ์ใดไว้ที่ พอร์ต ใดบ้าง

1.2 เขียนโปรแกรมเพื่อใช้ควบคุมอุปกรณ์

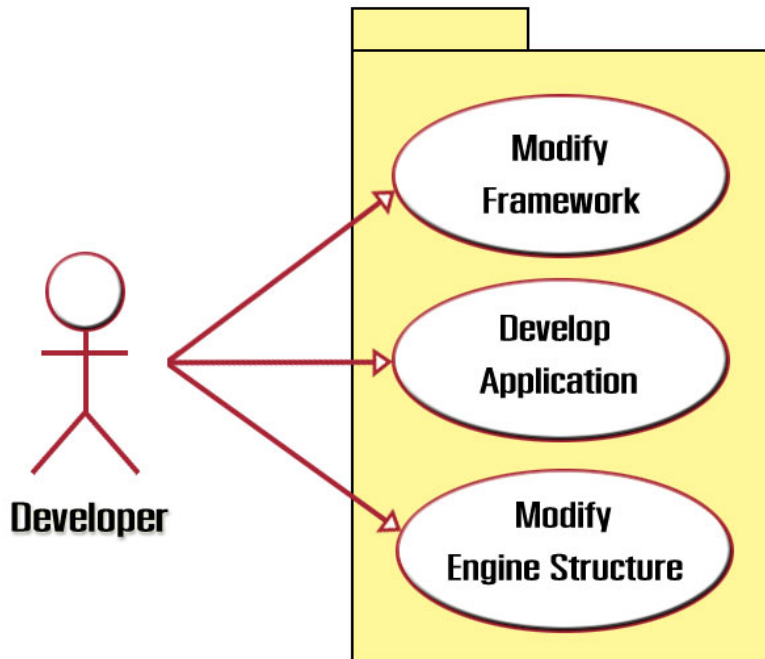
ผู้ใช้งาน สามารถเขียนโปรแกรม และดาวน์โหลดโปรแกรม ลงโดยใช้ฟังก์ชันจากชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ผ่าน แอปพลิเคชัน ที่คณะผู้จัดทำสร้างขึ้นมา เพื่อให้หุ่นยนต์สามารถทำงานตามโปรแกรม ใน การทำงานแบบอัตโนมัติได้

1.3 คุณสมบัติของอุปกรณ์และการทำงานของหุ่นยนต์

ผู้ใช้งาน สามารถดูสถานะของอุปกรณ์ต่างๆ ที่มาเชื่อมต่อกับชุดพัฒนา คอม86 ได้ตลอดเวลาผ่านทางพอร์ตอนุกรม โดยจะทราบทันทีว่าอุปกรณ์แต่ละตัวมีการรับค่าหรือส่งค่าอะไรอยู่ เพื่อให้ผู้ใช้งานสามารถทราบถึงการทำงานของอุปกรณ์ และทำการตรวจสอบและแก้ไขโปรแกรม ให้เป็นไปอย่างถูกต้องได้

1.4 ควบคุมการทำงานของหุ่นยนต์โดยตรง

ผู้ใช้งาน สามารถควบคุมอุปกรณ์ที่เชื่อมต่อกับชุดพัฒนา คอม86 ได้โดยตรงในลักษณะของการทำงานโดยตรง และผู้ใช้งานจะควบคุมอุปกรณ์ต่างๆผ่านทาง คีย์บอร์ด โดยตรง หรือ ผู้ใช้งาน ควบคุมอุปกรณ์โดยใช้ฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ที่มีอยู่มาสั่งงานผ่านทางคอมพิวเตอร์โดยตรงได้



รูปที่ 4 ยูสเคสไดอะแกรมของผู้พัฒนา

2 ผู้นำไปพัฒนา (Developer)

2.1 แก้ไขและพัฒนา ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์

ผู้พัฒนา สามารถแก้ไขและเพิ่มเติมฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ที่คณะผู้จัดทำสร้างขึ้นเพื่อสามารถนำฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงานอุปกรณ์ ไปใช้ตามความต้องการที่หลากหลายขึ้น และ เพื่อให้สามารถรองรับโมดูลของอุปกรณ์ชนิดใหม่ที่เพิ่มขึ้นมาได้ในอนาคต

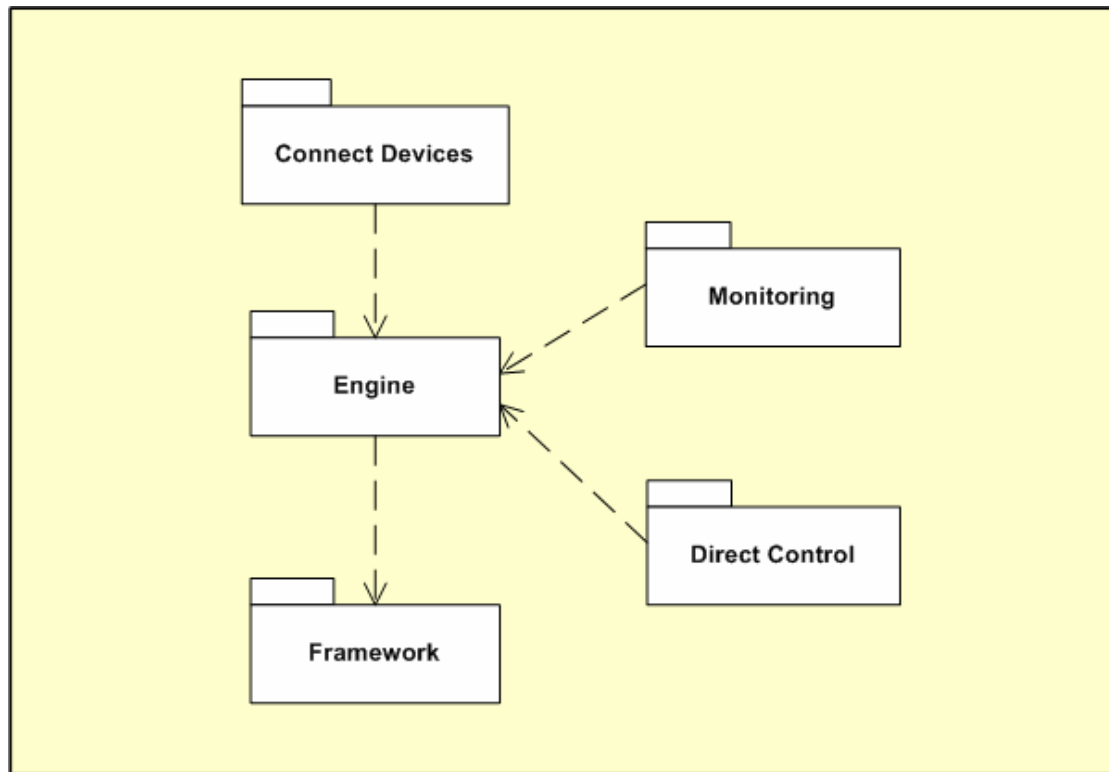
2.2 พัฒนาและเปลี่ยนแปลง GUI

ผู้พัฒนา สามารถแก้ไขและพัฒนา แอปพลิเคชัน ที่ใช้งานฟังก์ชัน ชุดคำสั่งสำหรับควบคุมและสั่งงาน อุปกรณ์ ที่คณะผู้จัดทำสร้างขึ้น ซึ่งอาจแก้ไขรูปแบบของ GUI หรือการทำให้เป็น แอปพลิเคชัน ใน pocket PC เพื่อควบคุมอุปกรณ์ที่เชื่อมต่อกับชุดพัฒนา คอม86 ได้

2.3 แก้ไขและปรับเปลี่ยนค่าของโปรแกรมควบคุมหลัก

ผู้พัฒนา สามารถแก้ไขโครงสร้างทาง Hardware บางอย่างของชุดพัฒนา คอม86ได้ เช่น เมมโมรี่ที่จองไว้สำหรับ ผู้ใช้งาน โปรแกรม การแก้ไขตำแหน่งการเก็บค่าในรีจิสเตอร์ต่างๆ หรือ การปรับปรุงแก้ไขแอดเดส เพื่อรองรับ โมดูลของอุปกรณ์ชนิดใหม่ที่เพิ่มขึ้นได้

คอมโพเนนต์ไดอะแกรม (Component Diagram)



รูปที่ 5 แผนภาพคอมโพเนนต์ของโครงการ

จากแผนภาพคอมโพเนนต์ที่กำหนดไว้จะทำการแบ่งหมวดหมู่ของโปรแกรมออกเป็น ส่วน ๆ โดยแต่ละส่วนจะมีการเรียกใช้อีกส่วนต่างกันไปตามลำดับ จากคอมโพเนนต์ไดอะแกรมที่ออกแบบขึ้นนั้น

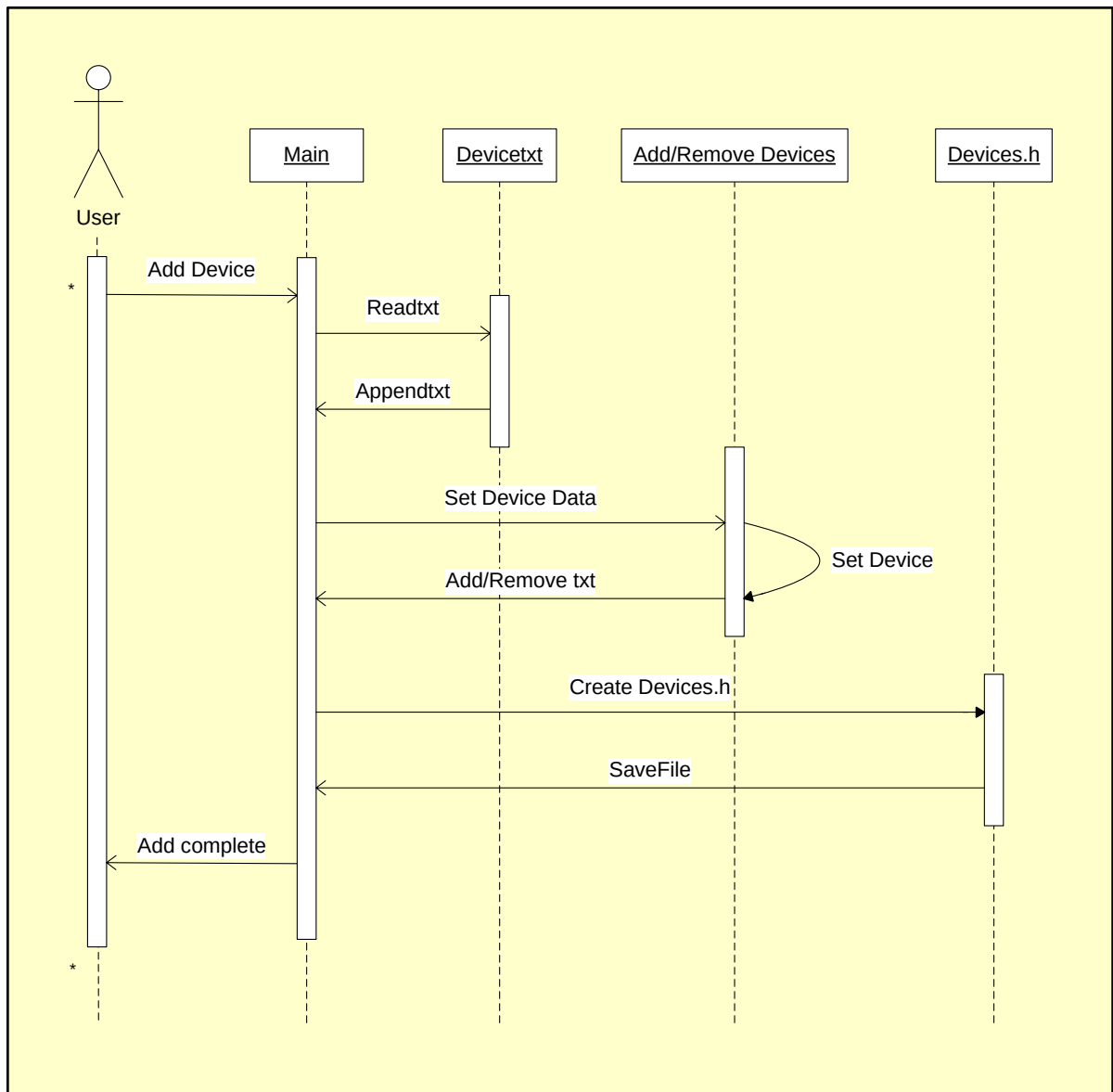
โปรแกรมหลักจะถูกแบ่งออกเป็น 5 ส่วนหลัก ดังนี้

- ส่วนของชุดคำสั่งที่เตรียมไว้สำหรับเรียกใช้ควบคุมอุปกรณ์ (Framework)
- ส่วนของโปรแกรมช่วยเหลือการทำงานโดยอัตโนมัติ (Engine)
- ส่วนของโปรแกรมที่ช่วยในการติดตั้งอุปกรณ์ (Connect Devices)
- ส่วนควบคุมหุ่นยนต์โดยตรง (Direct Control)
- ส่วนแสดงผลการทำงานจากหุ่นยนต์มายังคอมพิวเตอร์ (Monitoring)

ซีเควนส์ไดอะแกรม (Sequence Diagram) ของระบบ

ในส่วนนี้จะทำการแสดงถึงขั้นตอนการทำงานของโปรแกรมแอปพลิเคชันบนคอมพิวเตอร์ ในส่วนต่าง ๆ ตาม ความต้องการ ที่ได้กำหนดในขั้นต้น โดยแผนภาพจะแสดงรายละเอียดขั้นตอนการใช้งานโปรแกรม ควบคุม ผ่านทางแอปพลิเคชันบนคอมพิวเตอร์รวมถึงขั้นตอนการเรียกใช้งานออปเจ็กต์ต่าง ๆ ที่อยู่ในระบบ

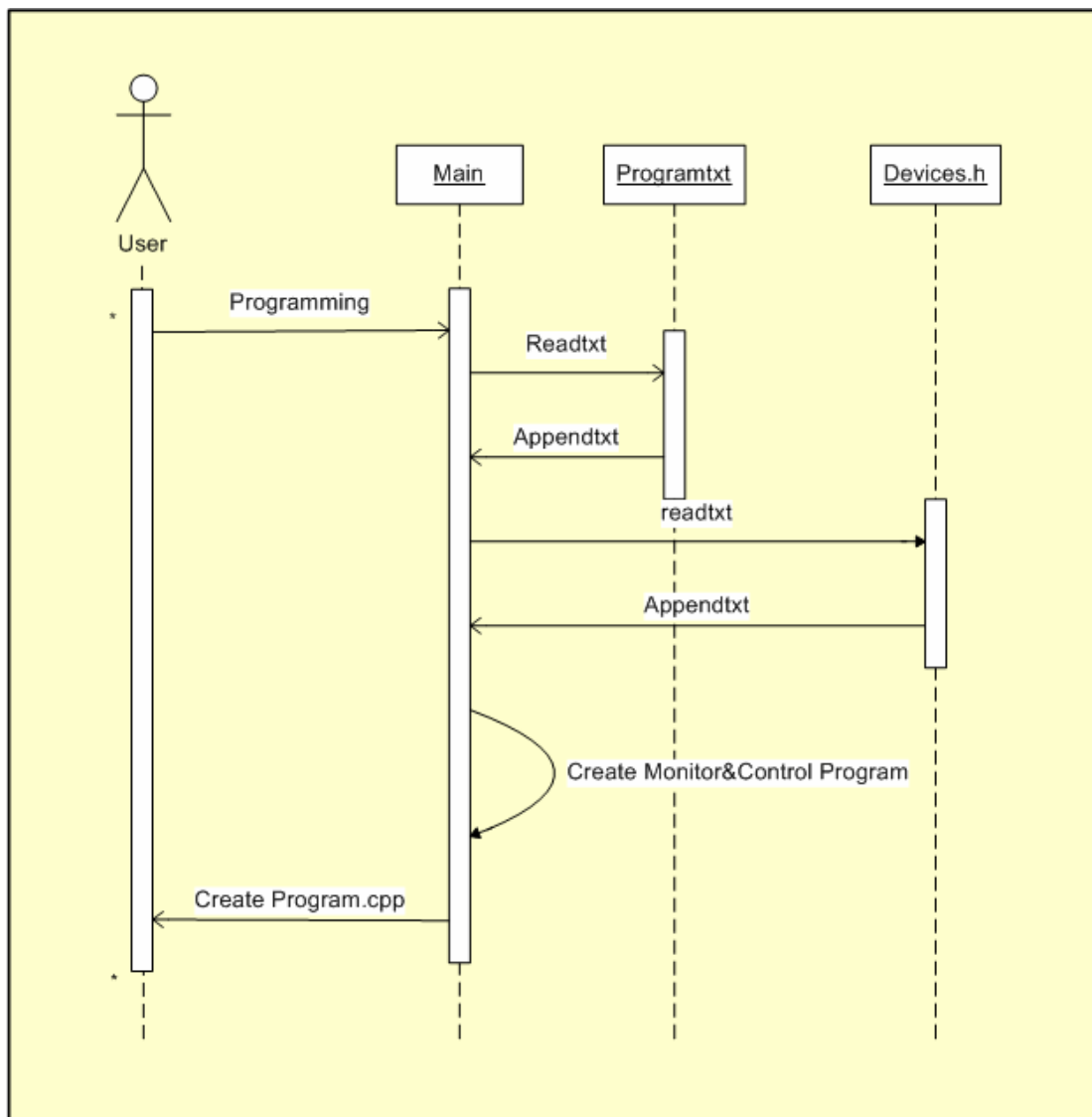
1 ติดตั้งอุปกรณ์กับบอร์ด คอม86 (Connect Devices)



รูปที่ 6 ซีเควนส์ไดอะแกรมของการติดตั้งอุปกรณ์ของผู้ใช้งาน

แผนภาพอธิบายการทำงานของโปรแกรมในส่วนการเชื่อมต่ออุปกรณ์กับคอม86 โดยผู้ใช้งานเริ่มจากเลือกที่จะติดตั้งอุปกรณ์โปรแกรมหลักจะทำการอ่านค่าไฟล์ข้อมูลอุปกรณ์ มาเก็บไว้เพื่อเตรียมทำการสร้างเฮดเดอร์ไฟล์ และหลังจากได้เลือกพอร์ตและอุปกรณ์จนครบแล้วโปรแกรมหลักจะทำการรวบรวมข้อมูลชนิดของอุปกรณ์พอร์ตที่ใช้แล้วทำการสร้างไฟล์ Devices.h ที่เป็นโลบารีของชุดคำสั่งที่เตรียมไว้สำหรับควบคุมอุปกรณ์ที่ติดตั้งไป

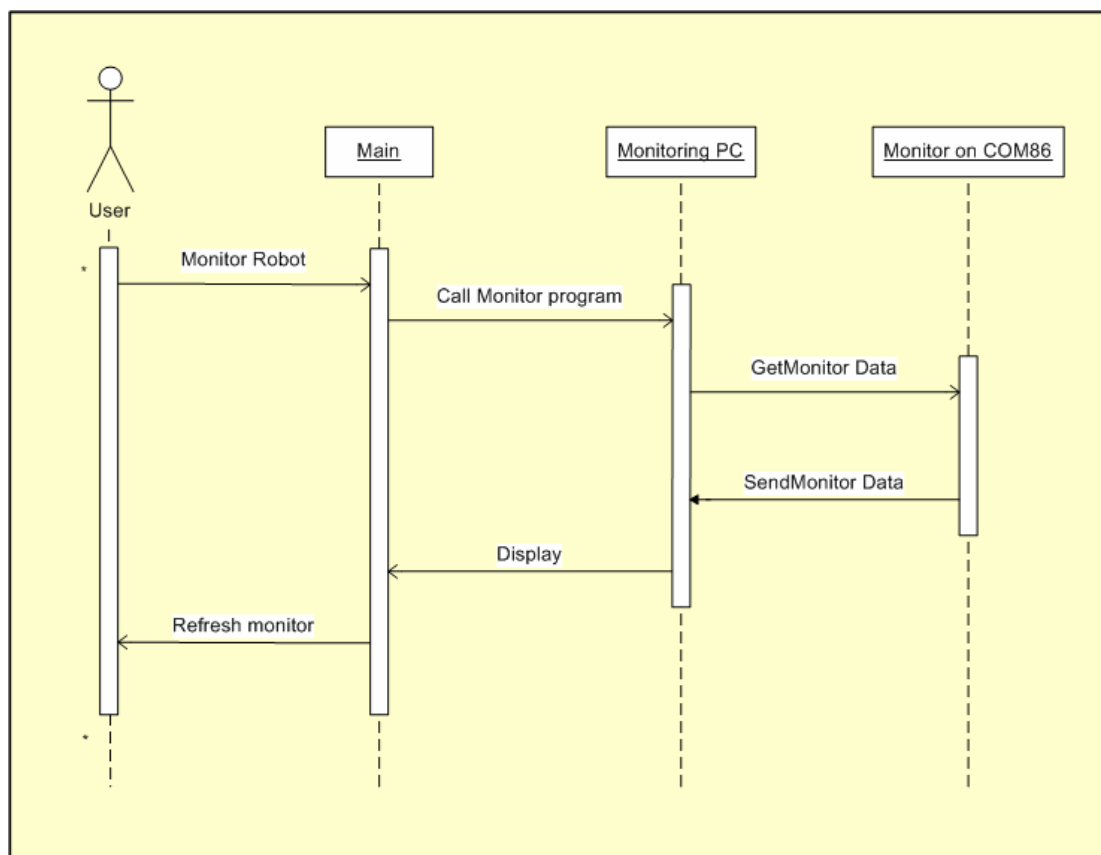
2 เขียนโปรแกรมเพื่อใช้ควบคุมอุปกรณ์ (Programming)



รูปที่ 7 ซีเควนส์ไดอะแกรมของการเขียนโปรแกรมสั่งงานอุปกรณ์ของผู้ใช้งาน

แผนภาพอธิบายการทำงานของผู้ใช้งานเรียกใช้โปรแกรมหลักเพื่อทำการเขียนโปรแกรม โดยมีขั้นตอนคือผู้ใช้งานจะทำการเรียกแอปพลิเคชันที่จะทำการเขียนโปรแกรม โดยโปรแกรมหลักจะเตรียมข้อมูลเพื่อทำการสร้างไฟล์ Program.cpp โดยการเรียกไฟล์ Devices.h แล้วทำการสร้างโครงสร้างโปรแกรมที่สนับสนุนการใช้งานระบบปฏิบัติการไมโครชิรุ่นที่2 พร้อมทั้งจะทำการสร้างออปเจคของอุปกรณ์ตามข้อมูลที่ได้เก็บบันทึกไว้ จากนั้นโปรแกรมหลักก็จะทำการเพิ่มโปรแกรมในส่วนของการส่งค่าควบคุมหุ่นยนต์ และส่วนโปรแกรมแสดงผลการทำงานไว้ในส่วนท้ายของโปรแกรมแล้วทำการเซฟไฟล์เพื่อให้ผู้ใช้งานนำไฟล์ที่ได้ไปสร้างไฟล์ Program.exe ด้วยการคอมไพล์บน โปรแกรมบอร์แลนด์ (Borland C)

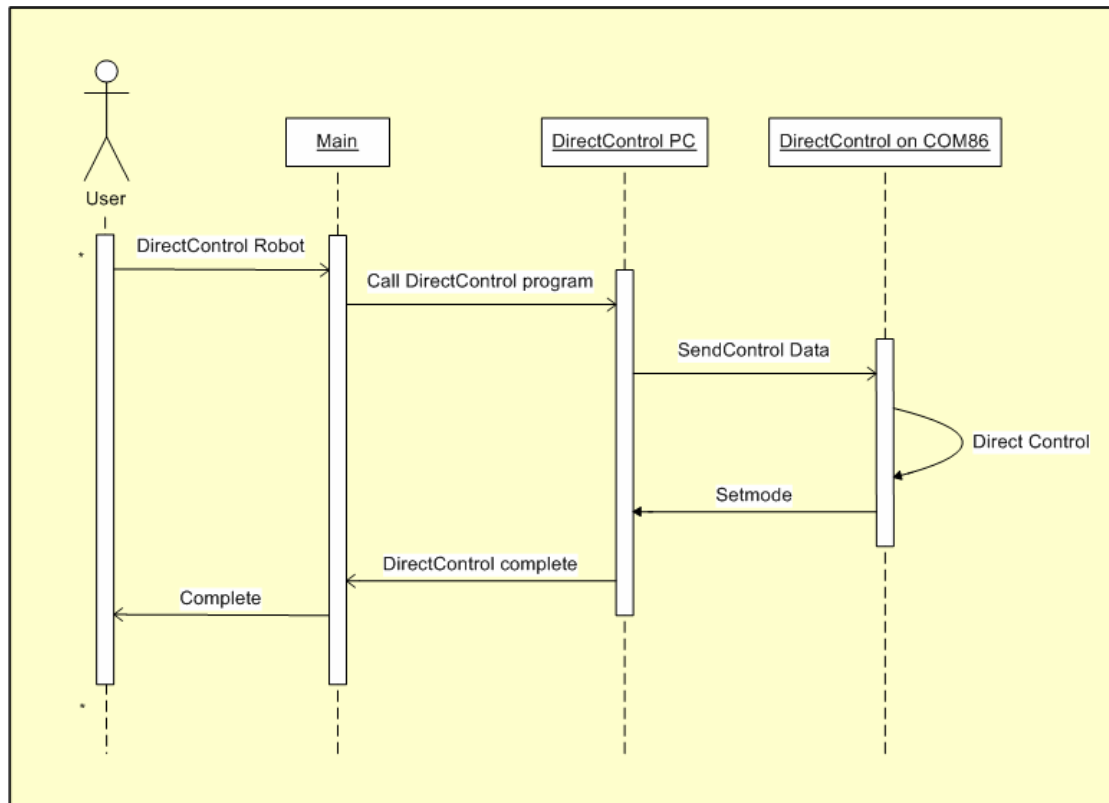
3 คุณสมบัติของอุปกรณ์และการทำงานของหุ่นยนต์ (Monitoring)



รูปที่ 8 ซีเควนส์ไดอะแกรมของการดูแลสถานะของอุปกรณ์และการทำงานของหุ่นยนต์

แผนภาพอธิบายการทำงานของผู้ใช้งานเรียกดูสถานะ การทำงานของอุปกรณ์ที่ติดตั้งอยู่บนหุ่นยนต์ โดยการเรียกดูค่าสถานะของหุ่นยนต์นั้นสามารถเรียกดูค่าผ่านทางแอปพลิเคชันโปรแกรมบนคอมพิวเตอร์ได้ โดยการเรียกดูสถานะการทำงานของหุ่นยนต์นั้น เริ่มจากโปรแกรมหลักจะทำการเรียกใช้งานโปรแกรมแสดงการทำงานที่ทำงานอยู่บนคอม86 ผ่านทางพอร์ตอนุกรม โดยโปรแกรมแสดงผลที่ทำงานอยู่บนคอม86 จะทำการดึงข้อมูลสถานะของอุปกรณ์บนคอม86 ส่งไปยังคอมพิวเตอร์ผ่านทางพอร์ตเดียวกัน หลังจากโปรแกรมหลักทำการรับค่าจากคอม86 แล้วจะทำการตีความจากข้อมูลที่ได้รับมาได้ แล้วทำการแสดงผลการทำงานออกทาง 모니터 โดยรายละเอียดของการแสดงผลนั้นสามารถดัดแปลงได้ตามความต้องการ โดยโปรแกรมออกแบบมาช่วยเหลือในการช่วยรับค่าและส่งผ่านข้อมูล จากคอม86 มายังคอมพิวเตอร์เท่านั้น

4 ควบคุมการทำงานของหุ่นยนต์โดยตรง (Direct control)



รูปที่ 9 ซีเควนส์ไดอะแกรมของการควบคุมการทำงานของหุ่นยนต์โดยตรง

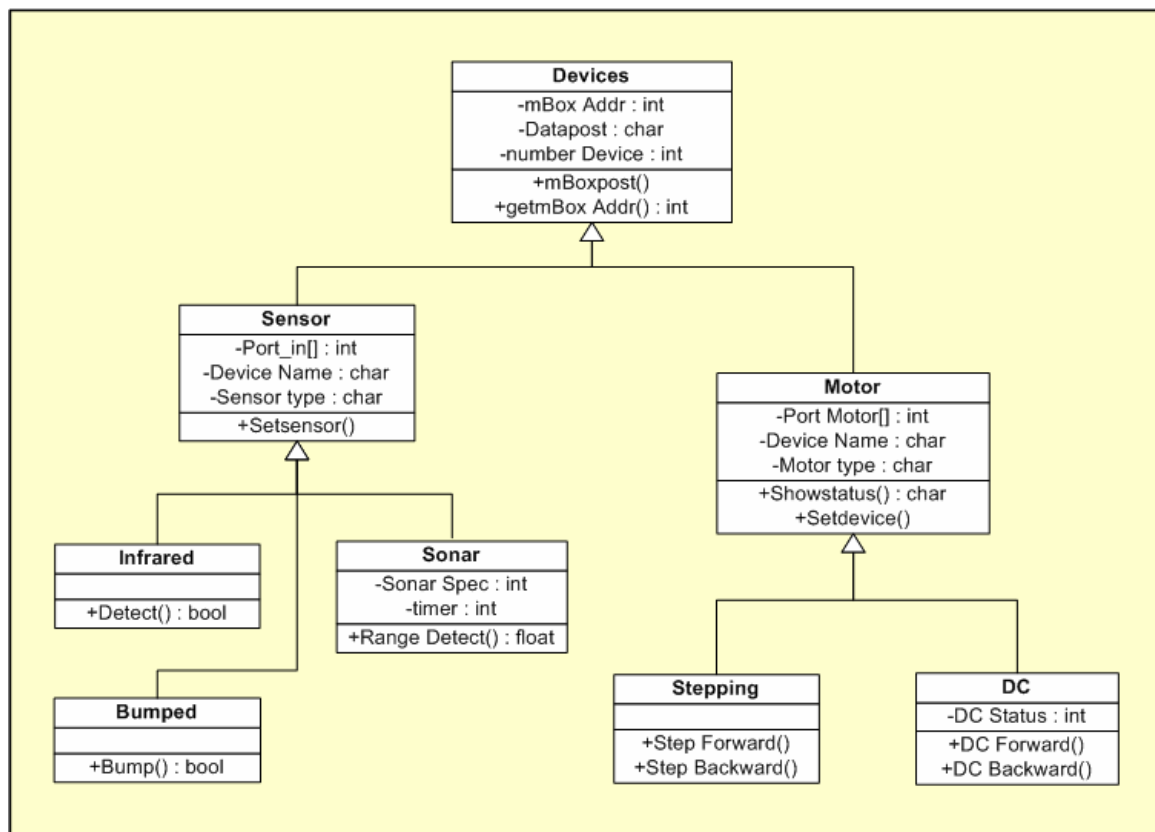
แผนภาพอธิบายการควบคุมหุ่นยนต์ โดยตรงจากคอมพิวเตอร์ ในที่นี้ผู้ใช้งานสามารถส่งค่าไปยังคอม86 เพื่อทำการแก้ไขควบคุมหรือเปลี่ยนแปลงการทำงานของหุ่นยนต์ ได้ตลอดเวลา โดยรายละเอียดขั้นตอนการทำงานของโปรแกรมเริ่มจากผู้ใช้งานเรียกใช้โปรแกรมหลักโดยต้องการที่จะทำการควบคุมอุปกรณ์บนตัวหุ่นยนต์ จากนั้นโปรแกรมหลักจะทำการเรียกโปรแกรมควบคุมการทำงานของอุปกรณ์ที่ทำงานอยู่บนคอม86 โดยการส่งค่าไปบอกหลังจากคอม86ได้เปลี่ยนโหมดการทำงานแล้ว ผู้ใช้งานก็จะสามารถควบคุมหุ่นยนต์จากคอมพิวเตอร์โดยตรงผ่านทางพอร์ตอนุกรมได้

คลาสไดอะแกรม (Class Diagram) ของระบบ

คลาสไดอะแกรมของระบบจะแบ่งออกเป็น 2 ส่วนใหญ่ๆคือ

- คลาสไดอะแกรมของระบบในส่วนของคุณค่าสั่งสำหรับควบคุมอุปกรณ์
- คลาสไดอะแกรมของระบบในส่วนของโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์

1 คลาสไดอะแกรมของระบบในส่วนของคุณค่าสั่งสำหรับควบคุมอุปกรณ์



รูปที่ 10 คลาสไดอะแกรมในส่วนของคุณค่าสั่งสำหรับควบคุมอุปกรณ์ (Framework)

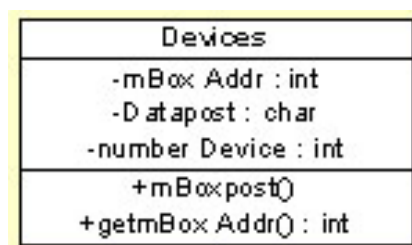
จากภาพเป็นการแสดงรายละเอียดของคลาสภายในโปรแกรมที่เป็นส่วนของคุณค่าสั่งที่ใช้สำหรับควบคุมและใช้งานอุปกรณ์ ที่ผู้ใช้งานได้ติดตั้งไว้ ทางคณะผู้จัดทำได้ออกแบบชุดคำสั่งเป็น 2 ส่วนหลักด้วยกันคือส่วนของอุปกรณ์ที่ทำการรับค่าเข้ามายังตัวประมวลผล กับส่วนของอุปกรณ์ที่ทำการส่งข้อมูลไปสั่งการ จากภาพจะเห็นส่วนของมอเตอร์ และส่วนของเซนเซอร์แล้วรวมถึงรายละเอียดของคลาสที่ย่อยลงมาโดยในแต่ละคลาสที่ออกแบบนั้น มีรายละเอียดภายใน ดังต่อไปนี้

- คลาสมอเตอร์ ประกอบด้วยคลาสสเตปป์มอเตอร์ และ ดีซีมอเตอร์
- คลาสเซนเซอร์ ประกอบด้วยคลาส อินฟราเรด , อัลตราโซนิก และสวิตช์แบบสัมผัส

โดยคลาสแม่ของโปรแกรมจะมีการเก็บค่าข้อมูลที่จำเป็นต่อการเรียกใช้งาน เช่น ชื่อของออปเจกของอุปกรณ์ พอร์ตที่กำหนดทางลอจิกคอลโดยผู้ใช้งาน และค่าแอดเดสที่กำหนด สำหรับการส่งค่าไปยังเมมโมรี่ของแต่ละทาสก์ของชุดคำสั่งที่จะทำการส่งค่าภายในแต่ละทาสก์ของระบบปฏิบัติการไมโครชี 2

ส่วนคลาสลูกที่ทำการอินเฮอร์ลิเตนส์ (Inhabitant) มาจะประกอบด้วยฟังก์ชัน ที่เตรียมไว้สำหรับ เรียกใช้เพื่อทำการควบคุมอุปกรณ์ โดยประกอบด้วยชุดคำสั่งสำหรับ ดิจิโมเตอร์, สเตปปีงมอเตอร์, อิน ฟาเรดเซนเซอร์, อัลตราโซนิก และสวิตช์สัมผัส ที่กำหนดเป็นอุปกรณ์พื้นฐาน

1.1 ลักษณะของคลาส Devices

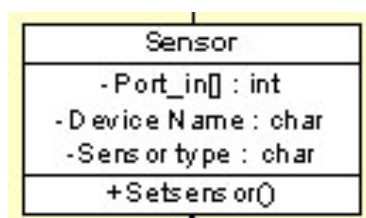


รูปที่ 11 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Devices

คลาส Devices เป็นคลาสพื้นฐานสำหรับอุปกรณ์ทุกชนิด โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ mBoxAddr เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของเมล์บ็อก ที่ใช้ในการส่งค่าข้อมูลควบคุมอุปกรณ์ระหว่างทาสก์ต่างในโปรแกรม
- แอตทริบิวต์ Datapost เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าข้อมูลที่ส่งไปควบคุมอุปกรณ์แต่ละชนิด
- แอตทริบิวต์ number Device เป็นแอตทริบิวต์ชนิดตัวเลข ที่ใช้เก็บข้อมูลจำนวนอุปกรณ์แต่ละชนิดว่าอุปกรณ์ชนิดนั้นมีจำนวนเท่าไร
- เมธอด mBoxpost() เป็นเมธอดที่ใช้สำหรับส่งข้อมูลระหว่างกันในแต่ละทาสก์เพื่อนำข้อมูลเหล่านั้นไปใช้ควบคุมอุปกรณ์
- เมธอด getmBoxAddr() เป็นเมธอดที่ใช้สำหรับเรียกให้ส่งค่าแอตทริบิวต์ mBoxAddr กลับมา โดยมีการส่งค่ากลับมาเป็นตัวเลข

1.2 ลักษณะของคลาส Sensor

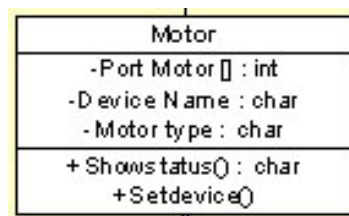


รูปที่ 12 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Sensor

คลาส Sensor เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดที่เป็นอินพุตต่างๆ ที่สืบทอดมาจากคลาส Device โดยมีส่วนประกอบดังนี้

- แอดทรีบิวต์ Port_in[] เป็นแอดทรีบิวต์ชนิดชุดของตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของอุปกรณ์โดยตำแหน่งของอุปกรณ์นี้จะใช้ในการอ้างอิงสำหรับควบคุม(รับข้อมูล)อุปกรณ์
- แอดทรีบิวต์ DviceName เป็นแอดทรีบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าชื่อที่ผู้ใช้ตั้งให้กับอุปกรณ์ต่างๆ
- แอดทรีบิวต์ Sensortype เป็นแอดทรีบิวต์ชนิดตัวอักษร ที่ใช้เก็บข้อมูลระบุชนิดของอุปกรณ์เซนเซอร์ว่าเป็นเซนเซอร์ชนิดใด
- เมททอด Setsensor() เป็นเมททอดที่ใช้สำหรับกำหนดค่าเริ่มต้นและกำหนดค่าแอดทรีบิวต์ต่างๆของอุปกรณ์เซนเซอร์แต่ละชนิด

1.3 ลักษณะของคลาส Motor

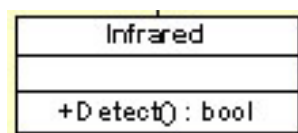


รูปที่ 13 รายละเอียดแอดทรีบิวต์และเมททอดของคลาส Motor

คลาส Motor เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดที่เป็นเอาต์พุตต่างๆ ที่สืบทอดมาจากคลาส Device โดยมีส่วนประกอบดังนี้

- แอดทรีบิวต์ PortMotor[] เป็นแอดทรีบิวต์ชนิดชุดของตัวเลข ที่มีไว้สำหรับเก็บค่าตำแหน่งของอุปกรณ์โดยตำแหน่งของอุปกรณ์นี้จะใช้ในการอ้างอิงสำหรับควบคุม(ส่งข้อมูล)อุปกรณ์
- แอดทรีบิวต์ DeviceName เป็นแอดทรีบิวต์ชนิดตัวอักษร ที่มีไว้เก็บค่าชื่อที่ผู้ใช้ตั้งให้กับอุปกรณ์ต่างๆ
- แอดทรีบิวต์ Motortype เป็นแอดทรีบิวต์ชนิดตัวอักษร ที่ใช้เก็บข้อมูลระบุชนิดของอุปกรณ์มอเตอร์ว่าเป็นมอเตอร์ชนิดใด
- เมททอด Showstatus() เป็นเมททอดที่ใช้ส่งค่าการทำงานปัจจุบันของมอเตอร์มาให้ผู้ใช้ทราบ
- เมททอด Setdevice() เป็นเมททอดที่ใช้สำหรับกำหนดค่าเริ่มต้นและกำหนดค่าแอดทรีบิวต์ต่างๆของอุปกรณ์มอเตอร์แต่ละชนิด

1.4 ลักษณะของคลาส Infrared

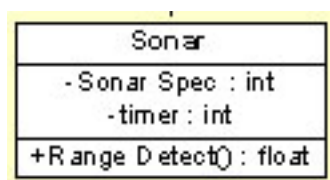


รูปที่ 14 รายละเอียดแอดทริบิวต์และเมทอดของคลาส Infrared

คลาส Infrared เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิดอินฟราเรดเซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- เมทอด Detect() เป็นเมทอดที่ใช้ส่งค่าการทำงานปัจจุบันของอินฟราเรดเซนเซอร์ มาให้ผู้นำไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตรรกะ

1.5 ลักษณะของคลาส Sonar

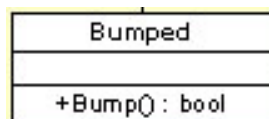


รูปที่ 15 รายละเอียดแอดทริบิวต์และเมทอดของคลาส Sonar

คลาส Sonar เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด อัลตราโซนิกเซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- แอดทริบิวต์ SonarSpec เป็นแอดทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลการเปิด-ปิดการใช้งาน อัลตราโซนิกเซนเซอร์
- แอดทริบิวต์ timer เป็นแอดทริบิวต์ชนิดตัวเลข ที่มีไว้เก็บค่าเวลาที่ใช้นับหลังจากมีการส่งค่าข้อมูลของ อัลตราโซนิกเซนเซอร์ เพื่อนำมาใช้มาคำนวณระยะทาง
- เมทอด RangeDetect() เป็นเมทอดที่ใช้คำนวณและส่งค่าระยะทางที่ อัลตราโซนิกเซนเซอร์ วัดได้ให้ผู้นำไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตัวเลขทศนิยม

1.6 ลักษณะของคลาส Bumped

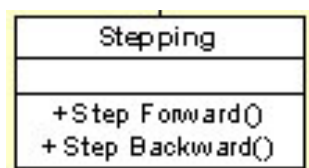


รูปที่ 16 รายละเอียดแอดทริบิวต์และเมทอดของคลาส Bumped

คลาส Bumped เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด บัมพ์เซนเซอร์ ที่สืบทอดมาจากคลาส Sensor โดยมีส่วนประกอบดังนี้

- เมทอด Bump() เป็นเมทอดที่ใช้ส่งค่าการทำงานปัจจุบันของ บัมพ์เซนเซอร์ มาให้ผู้นำไปใช้งาน โดยจะส่งค่ากลับมาเป็นค่าชนิดตรรกะ

1.7 ลักษณะของคลาส Stepping

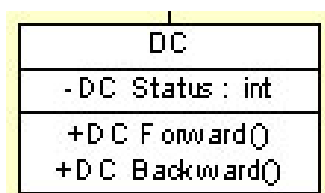


รูปที่ 17 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Stepping

คลาส Stepping เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด สเตปมอเตอร์ ที่สืบทอดมาจากคลาส Motor โดยมีส่วนประกอบดังนี้

- เมธอด StepForward() เป็นเมธอดที่ใช้ส่งคำสั่งงานให้ สเตปมอเตอร์ เคลื่อนที่ไปข้างหน้า
- เมธอด StepBackward() เป็นเมธอดที่ใช้ส่งคำสั่งงานให้ สเตปมอเตอร์ เคลื่อนที่กลับไปยังหลัง

1.8 ลักษณะของคลาส DC

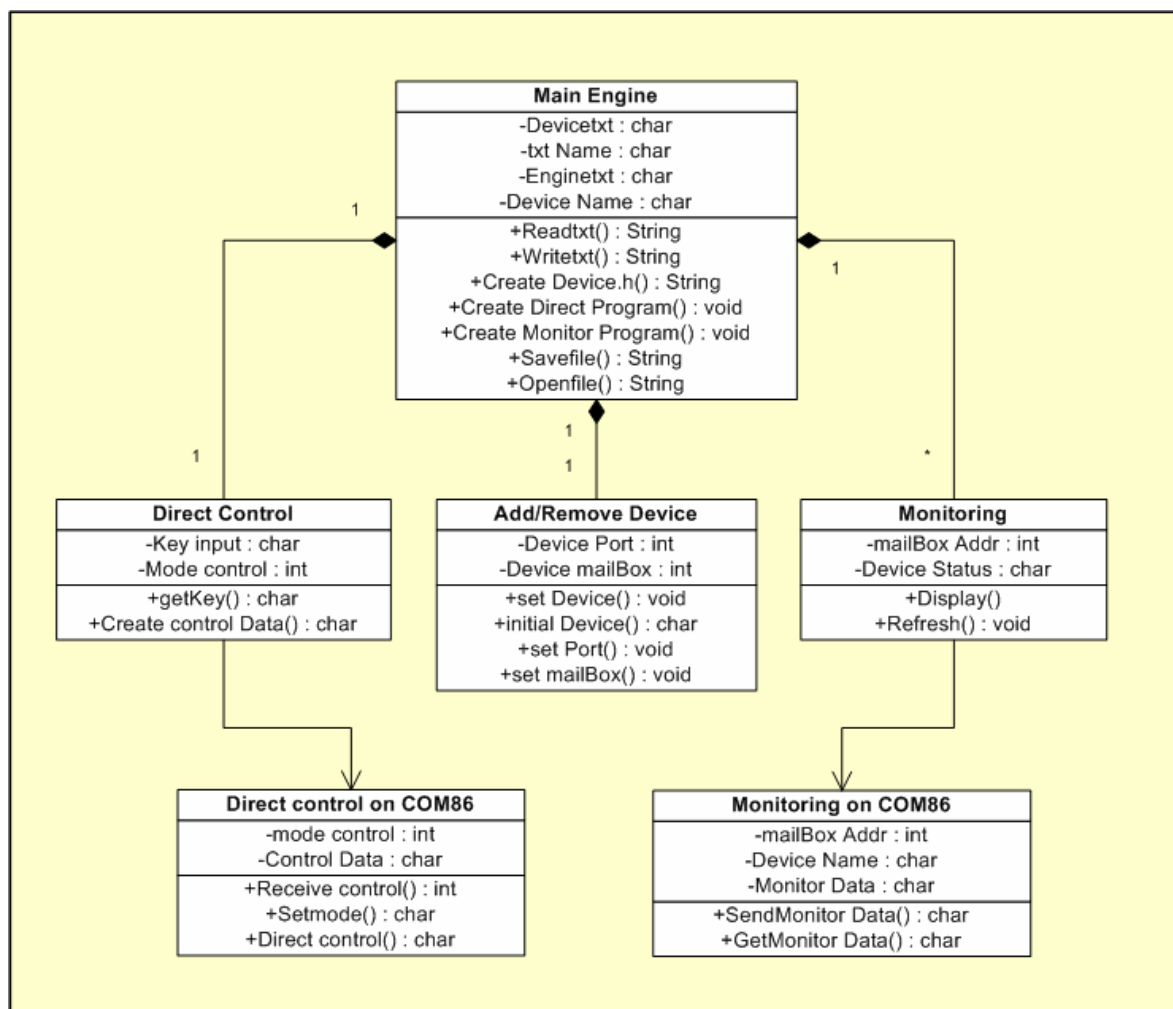


รูปที่ 18 รายละเอียดแอตทริบิวต์และเมธอดของคลาส DC

คลาส DC เป็นคลาสพื้นฐานสำหรับอุปกรณ์ชนิด ดิซีมอเตอร์ ที่สืบทอดมาจากคลาส Motor โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ DCstatus เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะการทำงานของ ดิซีมอเตอร์
- เมธอด DCForward() เป็นเมธอดที่ใช้ส่งคำสั่งงานให้ ดิซีมอเตอร์ เคลื่อนที่ไปข้างหน้า
- เมธอด DCBackward() เป็นเมธอดที่ใช้ส่งคำสั่งงานให้ ดิซีมอเตอร์ เคลื่อนที่กลับไปยังหลัง

2 คลาสไดอะแกรมของระบบในส่วนของโปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์



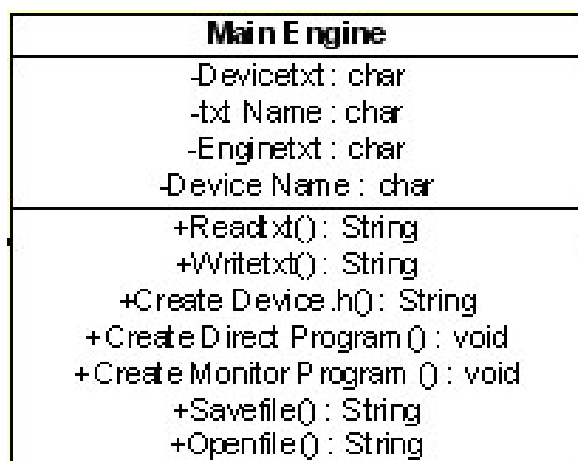
รูปที่ 19 คลาสไดอะแกรมในส่วน โปรแกรมช่วยเหลือสำหรับการใช้งานหุ่นยนต์ (Engine)

จากภาพเป็นการแสดงรายละเอียดของคลาสที่เป็นส่วนประกอบภายใน โปรแกรมช่วยเหลือ สำหรับการใช้งานหุ่นยนต์ ที่จะเป็นโปรแกรมที่จะทำการช่วยเหลือผู้ใช้งานให้สามารถจัดการเกี่ยวกับการ ติดตั้งอุปกรณ์ การเขียนโปรแกรม การสร้างโปรแกรมที่สามารถทำงานในแบบมัลติทาสก์ รวมถึงการเพิ่ม โปรแกรมในส่วนการแสดงผลการทำงานและส่งค่าควบคุมการทำงานของหุ่นยนต์ผ่านทางแอปพลิเคชัน บนคอมพิวเตอร์ โดยในแต่ละคลาสจะมามีการทำงานดังต่อไปนี้

- คลาสของส่วนโปรแกรมหลัก
- คลาสของการติดตั้งอุปกรณ์
- คลาสของการรับค่าการทำงานมาแสดงผลบนคอมพิวเตอร์
- คลาสของการส่งข้อมูลควบคุมการทำงานของหุ่นยนต์โดยตรง

โดยในส่วนของคลาสแสดงผลการทำงาน และคลาสควบคุมการทำงานนั้น จะแบ่งออกเป็นคลาสย่อย ที่ทำงานอยู่บนคอม86 ด้วยอีกส่วนหนึ่งเพื่อให้โปรแกรมหลักนั้น สามารถเรียกใช้ให้หุ่นยนต์สามารถส่ง ข้อมูลและรับข้อมูลมายังคอมพิวเตอร์ผ่านทางพอร์ตอนุกรม

2.1 ลักษณะของคลาส MainEngine



รูปที่ 20 รายละเอียดแอตทริบิวต์และเมธอดของคลาส MainEngine

คลาส MainEngine เป็นคลาสหลักของส่วนโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ Devicetxt เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลของอุปกรณ์ต่างๆ
- แอตทริบิวต์ txtName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลโปรแกรมหลักที่สร้างจากโปรแกรมช่วยเหลือผู้ใช้
- แอตทริบิวต์ Enginetxt เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของแฟ้มข้อมูลของส่วนประกอบของโปรแกรมช่วยเหลือผู้ใช้
- แอตทริบิวต์ DeviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของอุปกรณ์ต่างๆ
- เมธอด Readtxt() เป็นเมธอดที่ใช้อ่านแฟ้มข้อมูลเพื่อนำมาประมวลผลสร้างโปรแกรมหลัก
- เมธอด Writetxt() เป็นเมธอดที่ใช้เขียนค่าลงไปแฟ้มข้อมูลโปรแกรมหลัก
- เมธอด CreateDevice.h() เป็นเมธอดที่ใช้สร้างแฟ้มข้อมูลแบบเฮดเดอร์ ของอุปกรณ์ที่ผู้ใช้เลือก
- เมธอด CreateDirectProgram() เป็นเมธอดที่ใช้สร้างส่วนประกอบแฟ้มข้อมูลโปรแกรมหลักในส่วนทาสก์ที่ใช้ควบคุมอุปกรณ์โดยตรงจากผู้ใช้
- เมธอด CreateMonitorProgram() เป็นเมธอดที่ใช้สร้างส่วนประกอบแฟ้มข้อมูลโปรแกรมหลักในส่วนทาสก์ที่ใช้แสดงผลสถานะการทำงานของอุปกรณ์ต่างๆ
- เมธอด Savefile() เป็นเมธอดที่ใช้บันทึกแฟ้มข้อมูลโปรแกรมหลักที่โปรแกรมช่วยเหลือสร้างขึ้นมาแล้วให้นำไปเขียนโปรแกรมในส่วนของทาสก์ผู้ใช้

- เมธอด Openfile() เป็นเมธอดที่ใช้เปิดแฟ้มข้อมูลโปรแกรมหลักที่ผู้ใช้เคยบันทึกไว้แล้ว

2.2 ลักษณะของคลาส DirectControl

Direct Control
-Key input : char -Mode control : int
+getKey() : char +Create control Data() : char

รูปที่ 21 รายละเอียดแอตทริบิวต์และเมธอดของคลาส DirectControl

คลาส DirectControl เป็นคลาสที่เป็นส่วนประกอบของคลาส MainEngine โดยจะทำงานในส่วนการควบคุมอุปกรณ์โดยตรงของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ Keyinput เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าอินพุตที่รับจากผู้ใช้โดยวิธีการกดคีย์บอร์ด
- แอตทริบิวต์ Modecontrol เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะ การทำงานของการควบคุมอุปกรณ์
- เมธอด getKey() เป็นเมธอดที่ใช้รับค่าข้อมูลจากผู้ใช้งานผ่านทางคีย์บอร์ด
- เมธอด CreatecontrolData() เป็นเมธอดที่ใช้สร้างข้อมูลที่ส่งไปควบคุมอุปกรณ์แต่ละชนิดโดยตรง

2.3 ลักษณะของคลาส Add/Remove Device

Add/Remove Device
-Device Port : int -Device mailBox : int
+set Device() : void +initial Device() : char +set Port() : void +set mailBox() : void

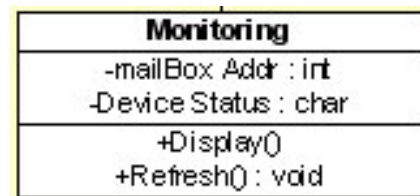
รูปที่ 22 รายละเอียดแอตทริบิวต์และเมธอดของคลาส Add/Remove Device

คลาส Add/Remove Device เป็นคลาสที่เป็นส่วนประกอบของคลาส MainEngine โดยจะทำงานในส่วนการติดตั้งอุปกรณ์ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ DevicePort เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่งที่ใช้อ้างอิงถึงอุปกรณ์ต่างๆที่ผู้ใช้เลือกไว้
- แอตทริบิวต์ DevicemailBox เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลเมล์บ็อกที่ใช้ส่งข้อมูลของแต่ละอุปกรณ์
- เมธอด setDevice() เป็นเมธอดที่ใช้กำหนดชนิดอุปกรณ์ที่ผู้ใช้เลือก

- เมθοอด initialDevice() เป็นเมθοอดที่ใช้กำหนดค่าเริ่มต้นของอุปกรณ์ที่ผู้ใช้เลือก
- เมθοอด setPort() เป็นเมθοอดที่ใช้กำหนดค่าตำแหน่งที่ใช้อ้างถึงอุปกรณ์ที่ผู้ใช้เลือก
- เมθοอด setmailBox() เป็นเมθοอดที่ใช้กำหนดค่าเมล็บ็อกให้กับอุปกรณ์ทุกตัวที่ผู้ใช้เลือก

2.4 ลักษณะของคลาส Monitoring



รูปที่ 23 รายละเอียดแอตทริบิวต์และเมθοอดของคลาส Monitoring

คลาส Monitoring เป็นคลาสที่เป็นส่วนประกอบของคลาส MainEngine โดยจะทำงานในส่วนการแสดงสถานะของอุปกรณ์ต่างให้ผู้ใช้ทราบทางจอแสดงผลของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอตทริบิวต์ mailBoxAddr เป็นแอตทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่งเมล็บ็อกที่ใช้ส่งข้อมูลของแต่ละอุปกรณ์
- แอตทริบิวต์ DeviceStatus เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะ การทำงานของอุปกรณ์แต่ละตัว
- เมθοอด Display() เป็นเมθοอดที่ใช้แสดงสถานะของอุปกรณ์ต่างทางจอแสดงผล
- เมθοอด Refresh() เป็นเมθοอดที่ใช้เปลี่ยนค่าที่แสดงผลใหม่

2.5 ลักษณะของคลาส Direct control on Com86

Direct control on COM86
-mode control : int -Control Data : char
+Receive control() : int +Setmode() : char +Direct control() : char

รูปที่ 24 รายละเอียดแอดทริบิวต์และเมธอดของคลาส Direct control on Com86

คลาส Direct control on Com86 เป็นคลาสย่อยที่เกี่ยวข้องกับคลาส DirectControl โดยจะทำงานในส่วนการควบคุมอุปกรณ์โดยตรงที่จะทำงานภายในคอม86 ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยมีส่วนประกอบดังนี้

- แอดทริบิวต์ modecontrol เป็นแอดทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะ การควบคุมอุปกรณ์ของโปรแกรมช่วยเหลือผู้ใช้งาน
- แอดทริบิวต์ ControlData เป็นแอดทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลที่ส่งไปสั่งงานอุปกรณ์ต่างๆโดยตรง
- เมธอด Receivecontrol() เป็นเมธอดที่รับค่าข้อมูลคำสั่งควบคุมอุปกรณ์จากโปรแกรมช่วยเหลือผู้ใช้งานที่อยู่บนคอมพิวเตอร์
- เมธอด Setmode() เป็นเมธอดที่รับค่าข้อมูลการกำหนดสถานะการควบคุมโปรแกรมช่วยเหลือผู้ใช้งานที่อยู่บนคอมพิวเตอร์มากำหนดให้กับโปรแกรมที่กำลังทำงานอยู่ที่คอม86
- เมธอด Directcontrol() เป็นเมธอดที่ใช้ส่งค่าข้อมูลคำสั่งควบคุมอุปกรณ์จากคอม86 ไปยังตัวอุปกรณ์นั้นๆ

2.6 ลักษณะของคลาส Monitoring on Com86

Monitoring on COM86
-mailBox Addr : int -Device Name : char -Monitor Data : char
+SendMonitor Data() : char +GetMonitor Data() : char

รูปที่ 25 รายละเอียดแอดทริบิวต์และเมธอดของคลาส Monitoring on Com86

คลาส Monitoring on Com86 เป็นคลาสย่อยที่เกี่ยวข้องกับคลาส Monitoring โดยจะทำงานในส่วนการแสดงผลสถานะการทำงานของอุปกรณ์ที่จะทำงานภายในคอม86 ของโปรแกรมช่วยเหลือผู้ใช้งาน โดยจะส่งข้อมูลไปยังส่วนของคอมพิวเตอร์ โดยมีส่วนประกอบดังนี้

- แอดทริบิวต์ mailBoxAddr เป็นแอดทริบิวต์ชนิดตัวเลข ที่มีไว้สำหรับเก็บค่าข้อมูลตำแหน่ง

เมธ็บ็อกที่ใ้ส่งข้อมูลของแต่ละอุปกรณ์

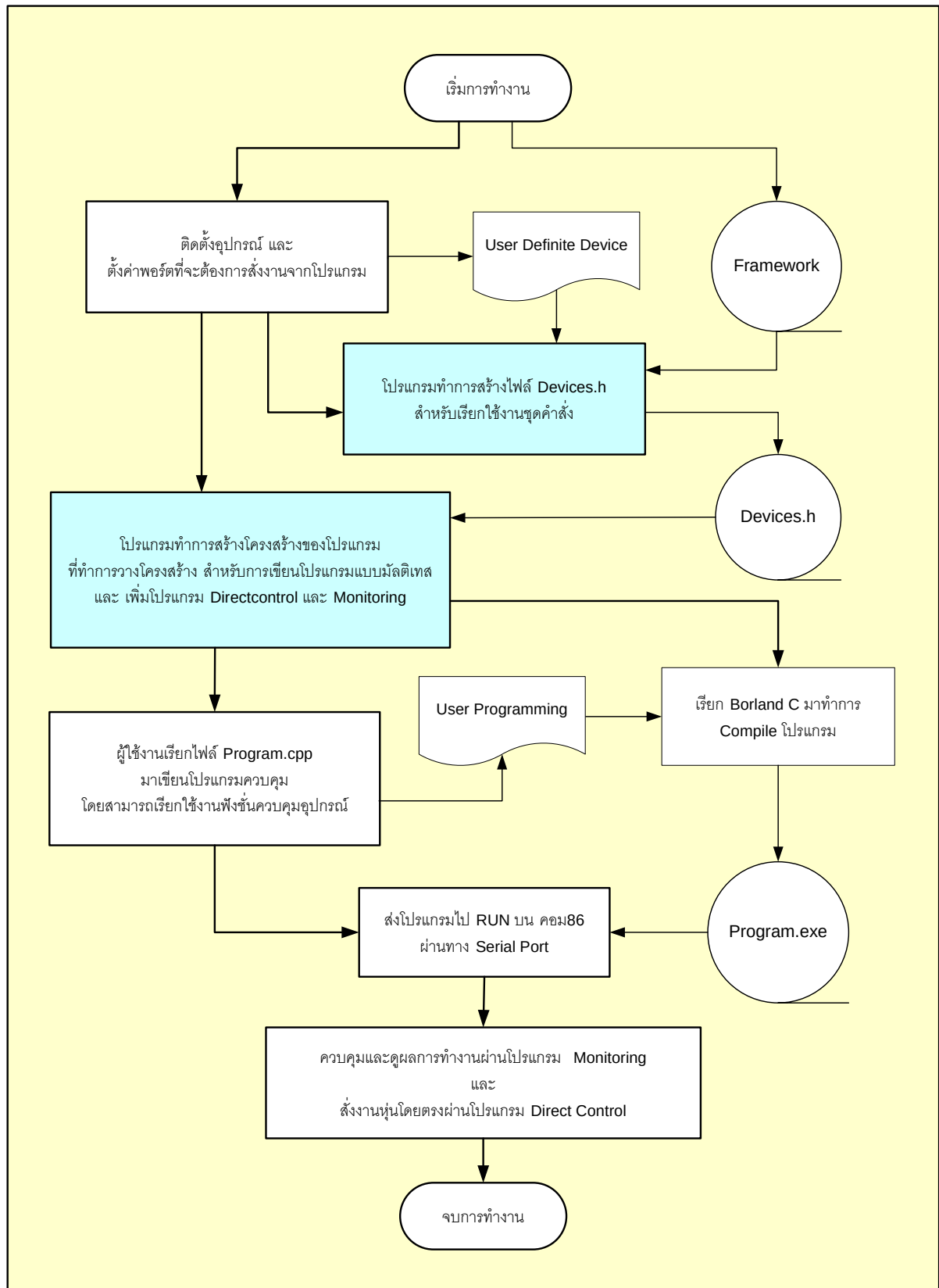
- แอตทริบิวต์ DeviceName เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าชื่อของอุปกรณ์ชนิดต่างๆที่ใ้ตั้งใ้

- แอตทริบิวต์ MonitorData เป็นแอตทริบิวต์ชนิดตัวอักษร ที่มีไว้สำหรับเก็บค่าข้อมูลสถานะ การทำงานของอุปกรณ์แต่ละตัวที่จะนำไปแสดงผล

- เมทธอด SendMonitorData() เป็นเมทธอดที่ใ้ส่งค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัวไปยังคอมพิวเตอร์เพื่อแสดงผลต่อไป

- เมทธอด GetMonitorData() เป็นเมทธอดที่ใ้รับค่าข้อมูลสถานะการทำงานของอุปกรณ์แต่ละตัวที่ทำงานอยู่ ณ.ปัจจุบัน

แผนภาพแสดงการทำงานของโปรแกรมจัดการระบบ (Design Flow)

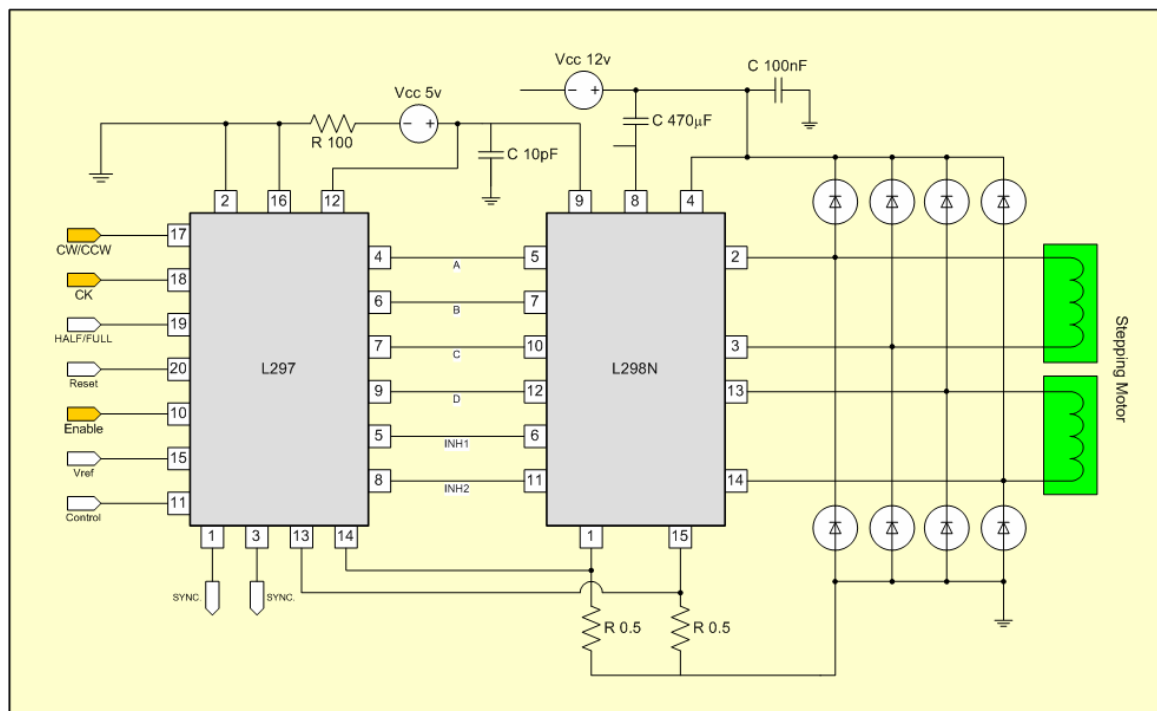


รูปที่ 26 แผนภาพแสดงการทำงานของโปรแกรมจัดการระบบ (Design Flow)

วงจรโมดูลของอุปกรณ์ (Devices Module)

เนื่องจากการเรียกใช้ชุดคำสั่งที่เตรียมไว้นั้น จะต้องมีการกำหนดค่าพอร์ตของอุปกรณ์ ที่จะต้องติดต่อกับไมโครคอนโทรลเลอร์อย่างถูกต้อง และเพื่อให้เข้ากันกับโปรแกรมช่วยเหลือผู้ใช้งานบนคอมพิวเตอร์ ทางคณะผู้จัดทำโครงการจึงได้ทำการออกแบบวงจรสำหรับใช้งานอุปกรณ์ และเป็นวงจรที่สามารถเรียกใช้งานชุดคำสั่งที่เตรียมการไว้อย่างถูกต้อง โดยไม่มีปัญหาแต่อย่างใด

1 สเตปมิ่งมอเตอร์

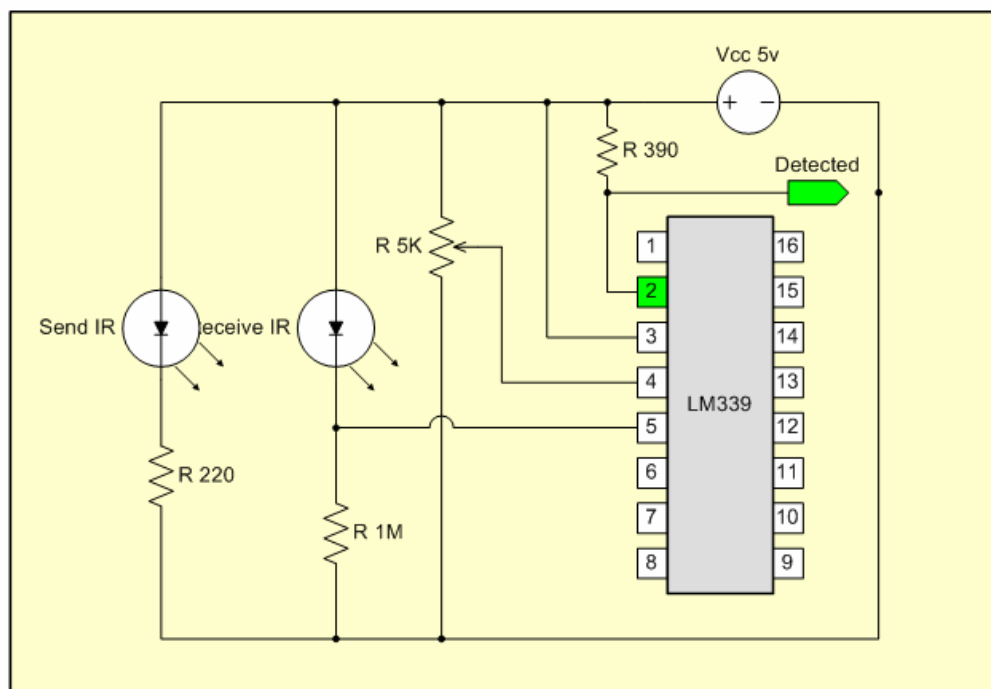
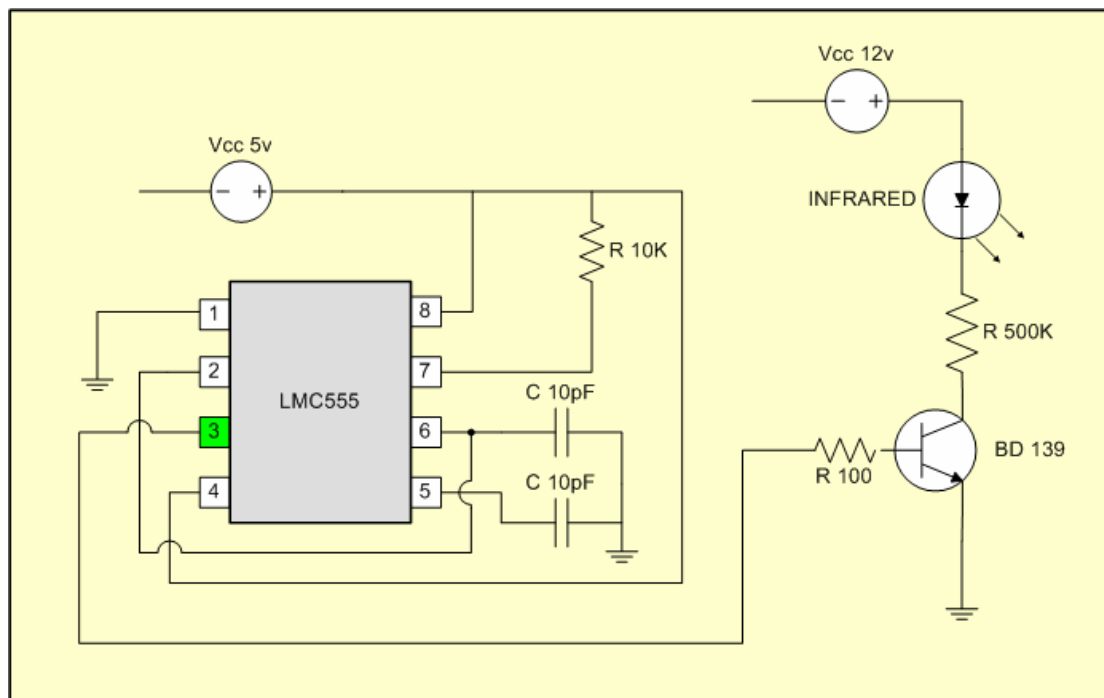


รูปที่ 27 แผนภาพแสดงโมดูลของการขับสเตปมอเตอร์ด้วย L297-L298N

โมดูลทำการขับเคลื่อนอุปกรณ์สเตปมอเตอร์ ตัวนี้ประกอบด้วยไอซี L297 ที่ทำหน้าที่เลื่อนบิต และทำการกำหนดทิศทาง การเลื่อนบิต รวมถึงการเพิ่มบิตที่ใช้งาน ซึ่งเป็นไอซีที่เหมาะสมกับการใช้งานควบคุมสเตปมอเตอร์อย่างมาก โดยขั้นตอนต่อไป ก็คือการเพิ่มกระแสในการขับเคลื่อนมอเตอร์ โดยทางคณะผู้จัดทำได้เลือก L298N ที่เป็นไอซีสำหรับการเพิ่มกระแสในการขับเคลื่อนมอเตอร์ที่รองรับการจ่ายไฟถึง 36 โวลต์ ซึ่งค่อนข้างจะครอบคลุมการใช้งานมอเตอร์เป็นอย่างมาก

โดยในส่วนการเพิ่มกระแส เพื่อขับเคลื่อนมอเตอร์ของไอซี L298N นั้นสามารถประยุกต์ใช้งานกับการขับเคลื่อนชนิดอื่น ๆ อีกด้วย โดยการเลือกขั้วอินพุตไปยัง ดีไซน์มอเตอร์ หรือรวมถึงการสร้างวงจรขับเคลื่อนมอเตอร์แบบ เฮลบริดจ์ (H-Bridge) สำหรับการควบคุมมอเตอร์ดีซี โดยจะไม่ยกตัวอย่างในที่นี้ โดยจะสามารถหาวิธีการจากภาคผนวกท้ายเล่มได้

2 อินฟราเรดเซนเซอร์



รูปที่ 28 - 29 แผนภาพแสดง โมดูลของการส่งอินฟราเรด

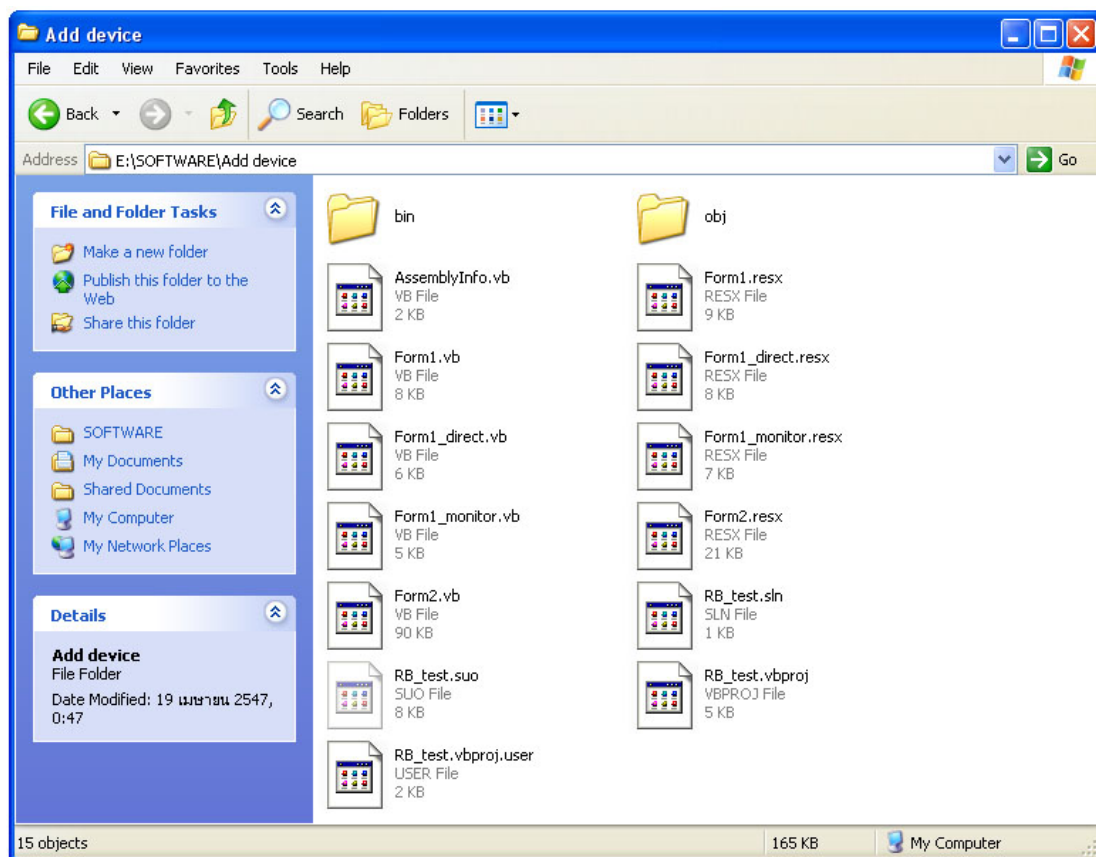
ส่วนประกอบของโปรแกรมต่างๆ

ส่วนประกอบของโปรแกรมทั้งหมดแบ่งออกเป็น 3 ส่วน

1. ส่วนของโปรแกรมติดตั้งอุปกรณ์
2. ส่วนของโปรแกรมสร้างเพิ่มข้อมูลโครงสร้างไมโครชิโอเอส
3. ส่วนของโปรแกรมที่เป็นการใช้งานไมโครชิโอเอส

โดยแต่ละส่วนจะมีลักษณะดังนี้

1. ส่วนของโปรแกรมติดตั้งอุปกรณ์ จะประกอบด้วยแฟ้มข้อมูลต่างๆ ดังรูป



รูปที่ 30 แฟ้มข้อมูลที่เป็นส่วนประกอบภายในโปรแกรมติดตั้งอุปกรณ์

โดยโปรแกรมการติดตั้งอุปกรณ์นี้เป็นโปรแกรมที่เขียนด้วยภาษา Visual Basic โดยจะประกอบด้วยแฟ้มข้อมูลแบบฟอร์มต่างๆที่ใช้สำหรับรับอินพุตจากผู้ใช้ ในการติดตั้งอุปกรณ์ (สามารถดูหน้าตาและการลักษณะการใช้งานของแบบฟอร์มต่างๆได้จากคู่มือการใช้งานโปรแกรม) โดยส่วนตัวโปรแกรมหลัก RB_test.vbproj จะทำหน้าที่ประมวลผลอินพุตต่างที่รับมาจากผู้ใช้ แล้วนำไปเก็บค่าเป็นเอาต์พุตไว้ที่ แฟ้มข้อมูล .txt เพื่อนำไปใช้ในโปรแกรมส่วนต่อไป

```
test_txt - Notepad
File Edit Format View Help

Devices Number 1 : Step Motor
Name OBJ is Step Motor
Port Select for Step Motor is 4 5 6 7 // สเตปมิ่งมอเตอร์ ชื่อ ออปเจค Step Motor ใช้งานพอร์ท 4,5,6,7

Devices Number 2 : Bump Sensor
Name OBJ is Bump Sensor
Port Select for Bump Sensor is 7

Devices Number 3 : DC Motor
Name OBJ is DC Motor
Port Select for DC Motor is 8 9 10 // ดิซีมอเตอร์ ชื่อ ออปเจค DC Motor ใช้งานพอร์ท 8,9,10

Devices Number 4 : Sonar
Name OBJ is Sonar
Port Select for Sonar is 10 11

Devices Number 5 : IR Sensor
Name OBJ is IR Sensor
Port Select for IR Sensor is 8 // อินฟราเรดเซนเซอร์ ชื่อ ออปเจค IR Sensor ใช้งานพอร์ท 8

Devices Number 6 : IR Sensor
Name OBJ is IR Sensor2
Port Select for IR Sensor2 is 9

Devices Number 7 : IR Sensor
Name OBJ is IR Sensor3
Port Select for IR Sensor3 is 10 // อินฟราเรดเซนเซอร์ ชื่อ ออปเจค IR Sensor3 ใช้งานพอร์ท 10

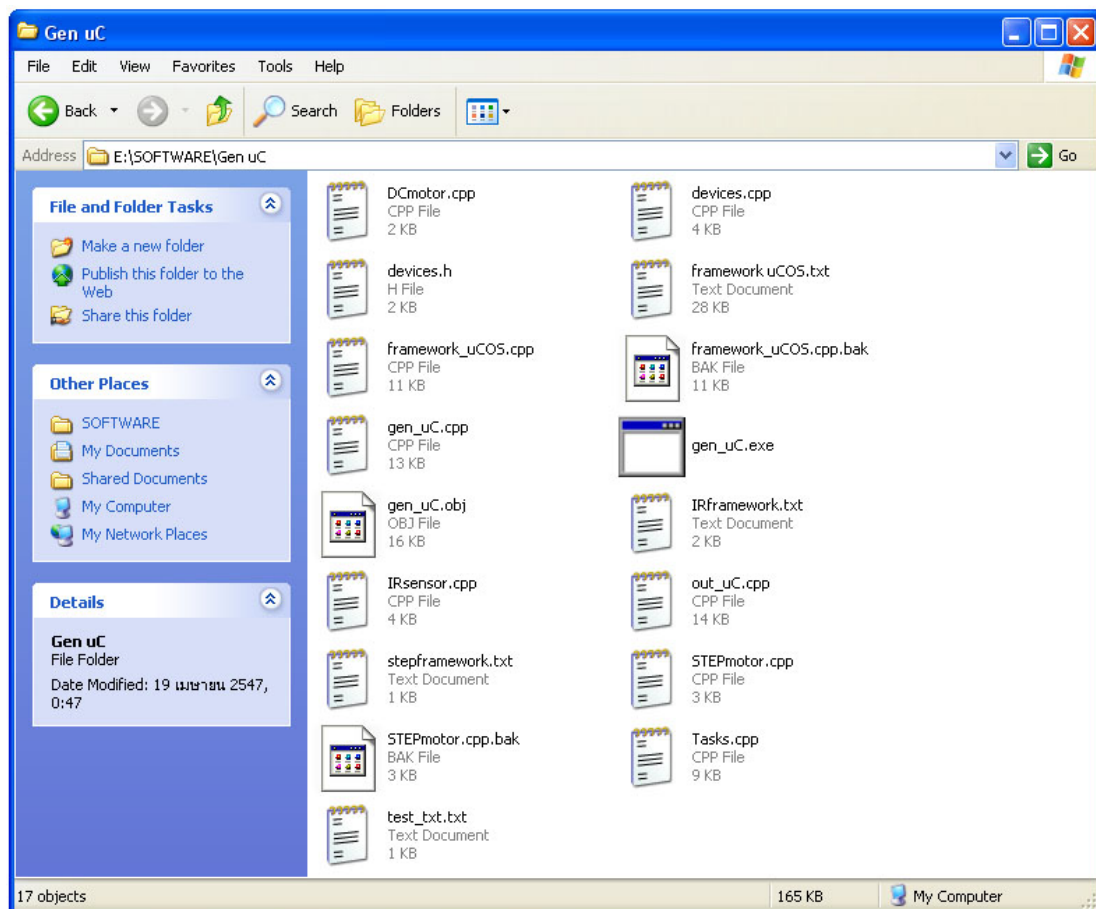
Devices Number 8 : IR Sensor
Name OBJ is IR Sensor4
Port Select for IR Sensor4 is 7
```

รูปที่ 31 แฟ้มข้อมูล .txt ที่เก็บไว้หลังจากที่ผู้ใช้งานติดตั้งอุปกรณ์

จากรูปที่ 31 เป็นสิ่งที่โปรแกรมติดตั้งอุปกรณ์เก็บไว้ในแฟ้มข้อมูลเอาท์พุท โดยจะประกอบด้วยส่วนต่างๆ ดังนี้

- หมายเลขของอุปกรณ์ โดยทั่วไปจะเกี่ยวข้องกับจำนวนอุปกรณ์
- ชนิดของอุปกรณ์
- ชื่อของอุปกรณ์ที่ผู้ใช้ตั้งให้ โดยจะนำไปใช้ในการตั้งชื่อออปเจคในไมโครซีไอเอสด้วย
- ค่าพอร์ทที่ผู้ใช้เลือกในการ ติดต่อกับอุปกรณ์ชนิดนั้นๆ

2. ส่วนของโปรแกรมสร้างแฟ้มข้อมูลโครงสร้างไมโครซีไอเอส จะประกอบด้วยแฟ้มข้อมูลต่างๆ ดังรูป



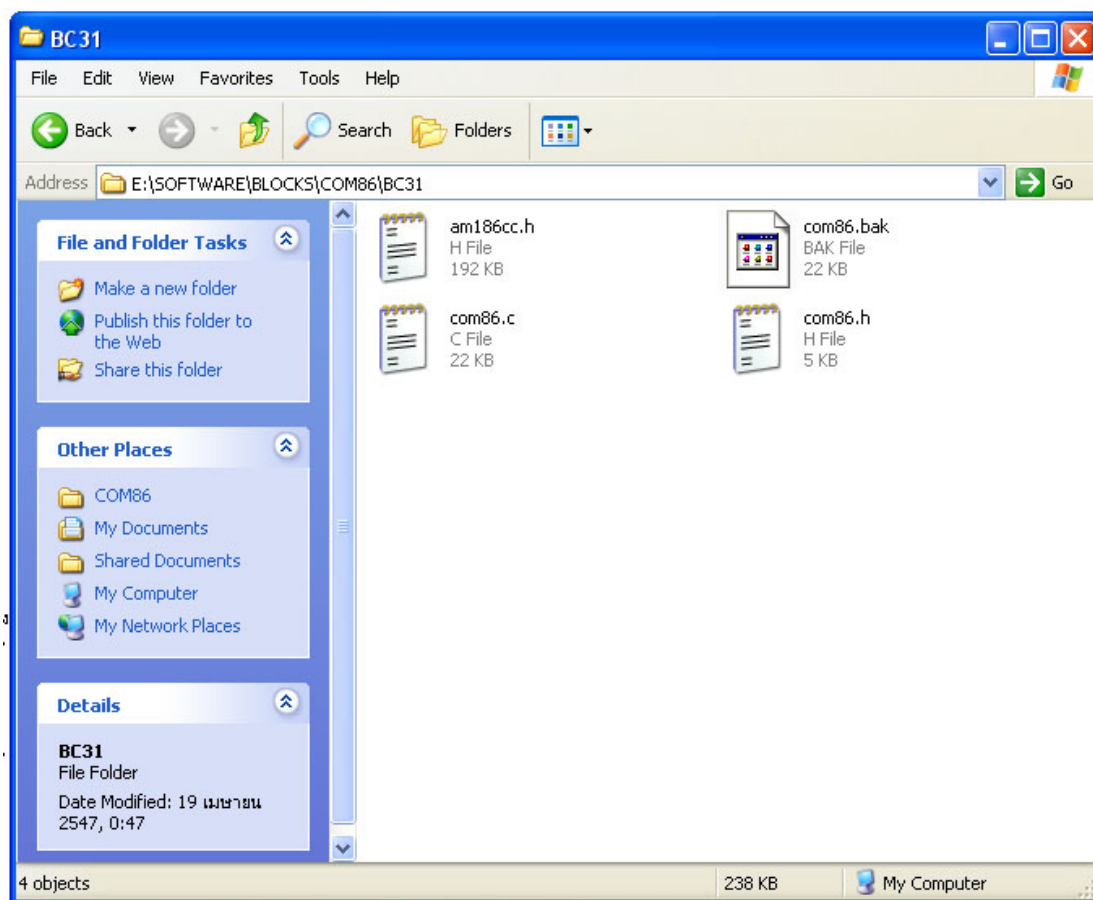
รูปที่ 32 เพิ่มข้อมูลที่เป็นส่วนประกอบภายในโปรแกรมสร้างโครงสร้างไมโครซีไอเอส 2

จะประกอบด้วยเพิ่มข้อมูล 4 ส่วนหลักๆ คือ

- ส่วนของเพิ่มข้อมูลอินพุตที่ได้จากโปรแกรมติดตั้งอุปกรณ์ ในรูปข้างบนคือเพิ่มข้อมูล test_txt.txt
- ส่วนของเพิ่มข้อมูลเอาต์พุตที่ได้จากโปรแกรมสร้างโครงสร้างไมโครซีไอเอส 2 ในรูปข้างบนคือเพิ่มข้อมูล out_uC.cpp
- โปรแกรมที่เป็นต้นแบบชุดคำสั่งที่จะนำไปเป็นส่วนประกอบของเพิ่มข้อมูลเอาต์พุต โดยจะมีชื่อเพิ่มข้อมูลเป็นชื่ออุปกรณ์ชนิดต่างๆ รวมถึงเพิ่มข้อมูล devices.h และ devices.cpp ด้วย
- ส่วนของเพิ่มข้อมูลโปรแกรมที่ใช้สร้างโครงสร้างไมโครซีไอเอสสำหรับผู้ใช้ คือเพิ่มข้อมูล gen_uC.exe ซึ่งมีรายละเอียดตัวโปรแกรมอยู่ที่ gen_uC.cpp โดยโปรแกรมนี้จะสร้างส่วนโครงสร้างหลักๆของไมโครซีไอเอส 2 มาเขียนลงไปในเพิ่มข้อมูลเอาต์พุต และจะนำเพิ่มข้อมูลในส่วนของเพิ่มข้อมูลชุดคำสั่งต่างๆ มาเปรียบเทียบกับเพิ่มข้อมูล .txt ที่ได้จากโปรแกรมติดตั้งอุปกรณ์ แล้วจะเลือกชุดคำสั่งที่เกี่ยวข้อง มาเขียนลงไปในเพิ่มข้อมูลเอาต์พุตด้วย ทำให้ได้เพิ่มข้อมูลเอาต์พุตที่เป็นโครงสร้างไม

โครชีโอเอสที่พร้อมทำงานกับอุปกรณ์ที่ผู้ใช้เลือกมาใช้ สำหรับให้ผู้ใช้นำไปเขียนโปรแกรมควบคุมอุปกรณ์ต่างๆต่อไป
สำหรับโครงสร้างของแฟ้มข้อมูลเอาต์พุตและแฟ้มข้อมูลชุดคำสั่งต่างๆจะอธิบายในส่วนถัดไป

3. ส่วนของโปรแกรมที่เป็นการใช้งานไมโครชีโอเอส จะประกอบด้วยแฟ้มข้อมูลหลายส่วนด้วยกัน โดยมีแฟ้มข้อมูลที่สำคัญ ดังนี้

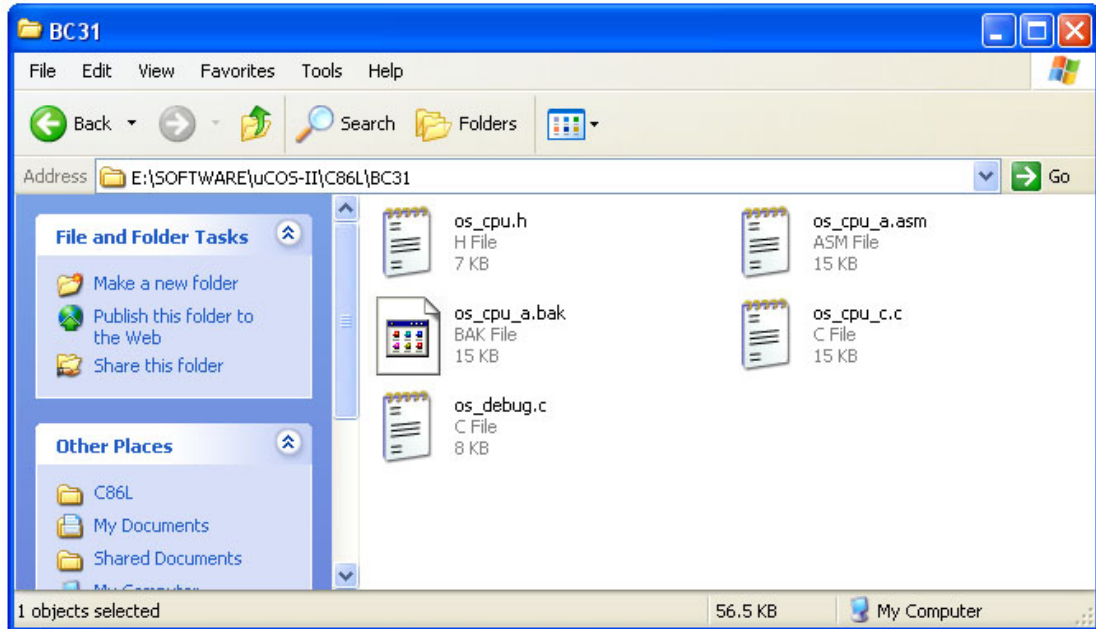


รูปที่ 33 แฟ้มข้อมูลส่วนการปรับแต่งให้ไมโครชีโอเอส 2 ใช้งานกับคอม86ได้

โดยจะประกอบ แฟ้มข้อมูล ดังนี้

- แฟ้มข้อมูล am186cc.h เป็นแฟ้มข้อมูลที่ใช้ปรับแต่งค่าตัวแปรต่างๆ ที่จะถูกอ้างถึงในการใช้งาน ไมโครชีโอเอสให้ตรงกับค่าการ register ของ CPU ของคอม86ซึ่งก็คือ Am186cc
- แฟ้มข้อมูล com86.h เป็นแฟ้มข้อมูลที่ใช้คู่กับแฟ้มข้อมูล com86.c โดยแฟ้มข้อมูล com86.h เป็นแฟ้มข้อมูล header สำหรับประกาศฟังก์ชันต่างๆ โดยที่จะอธิบายการทำงานของฟังก์ชันนั้นใน com86.c

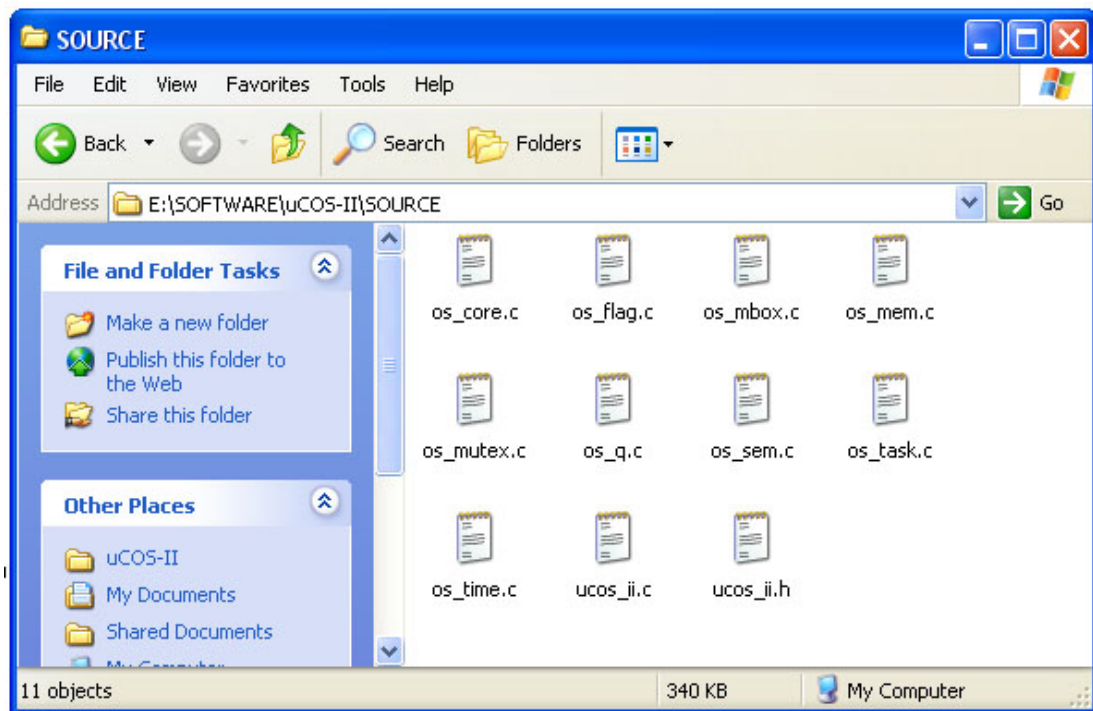
- เพิ่มข้อมูล com86.c เป็นเพิ่มข้อมูลที่ใช้อธิบายการทำงานของฟังก์ชันของไมโครซีไอเอส 2 โดยฟังก์ชันที่อยู่ภายใน com86.c นี้จะเป็นฟังก์ชันที่เกี่ยวข้องกับการทำงานของ register CPU ต่างๆ คือ ต้องปรับแก้เปลี่ยนแปลงตาม CPU ที่ใช้งาน



รูปที่ 34 เพิ่มข้อมูลส่วนการปรับแต่งการทำงานของไมโครซีไอเอส 2

โดยจะประกอบ เพิ่มข้อมูล ดังนี้

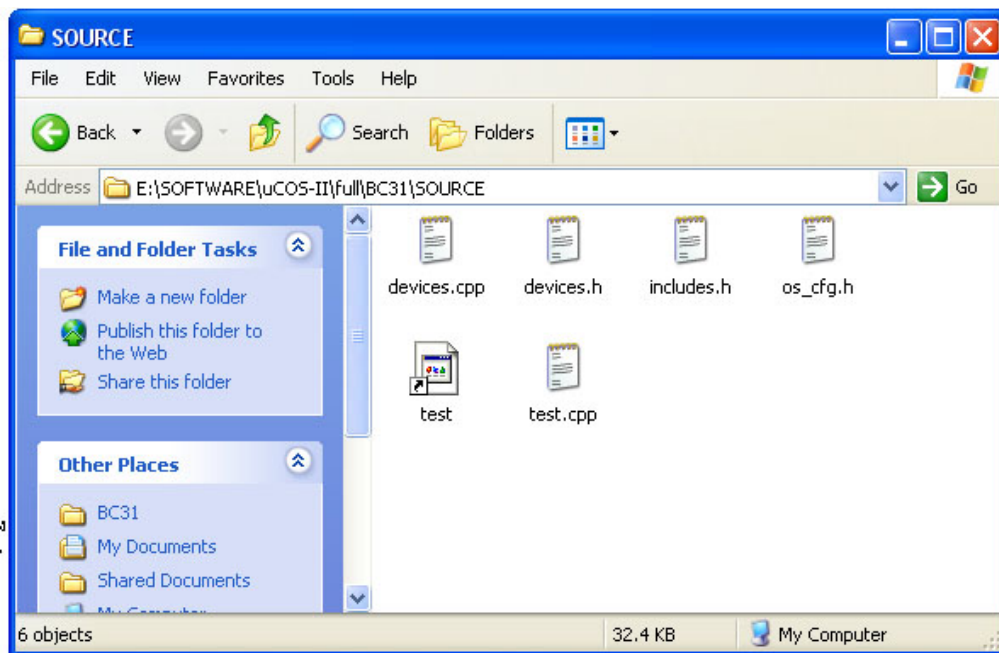
- เพิ่มข้อมูล os_cpu.h เป็นเพิ่มข้อมูลที่ใช้ประกาศลักษณะการทำงานต่างของ cpu เพื่อใช้ฟังก์ชันต่างทำงานสอดคล้องกับ ลักษณะพื้นฐาน cpu
- เพิ่มข้อมูล os_debug.c เป็นเพิ่มข้อมูลที่ใช้กำหนดค่าการใช้งานฟังก์ชันต่างๆของไมโครซีไอเอสว่าจะใช้งานในลักษณะใด
- เพิ่มข้อมูล os_cpu_c.c และ os_cpu_a.asm เป็นเพิ่มข้อมูลที่ใช้กำหนดลักษณะการทำงานของ ฟังก์ชันหลักของไมโครซีไอเอส โดยจะเป็นภาษา ซี และแอสเซมบลี ตามลำดับ โดยฟังก์ชันหลักเหล่านี้เปรียบเสมือน การทำงานที่แท้ของฟังก์ชันทั่วไปอื่นๆของไมโครซีไอเอส (ฟังก์ชันอื่นๆจะเรียกใช้ฟังก์ชันหลัก)



รูปที่ 35 เพิ่มข้อมูลส่วนการทำงานของฟังก์ชันของไมโครซีโอเอส 2

โดยจะประกอบ เพิ่มข้อมูลที่เก็บการทำงาน ของฟังก์ชันต่างๆที่เรียกใช้ในไมโครซีโอเอส 2 ซึ่งเป็นเพิ่มข้อมูล .C โดยแต่ละเพิ่มข้อมูลมีฟังก์ชันในส่วนการทำงานต่างๆที่ตรงกับชื่อเพิ่มข้อมูล เช่น เพิ่มข้อมูล os_task.c เป็นเพิ่มข้อมูลที่เก็บฟังก์ชันเกี่ยวกับการทำงานหลักๆของไมโครซีโอเอส ไม่ว่าจะเป็นการประกาศทาสก์ การควบคุมการทำงานต่างๆของทาสก์ หรือการดูค่าต่างๆของทาสก์ เป็นต้น ซึ่งการศึกษว่าแต่ละฟังก์ชันทำงานอย่างไรสามารถ ศึกษาจากการทำงานของไมโครซีโอเอส 2

เพิ่มข้อมูลที่กล่าวมาในส่วนของโปรแกรมที่เป็นการใช้งานไมโครซีโอเอส ข้างบนทั้งหมดนั้นจะเป็นส่วนประกอบ สำหรับการ compile โปรแกรมที่เราต้องการซึ่งจำเป็นต้องมีและโดยส่วนมากจะเป็นโปรแกรมที่มีค่าคงที่ (ในกรณีใช้ CPU ชนิดเดียวกัน) ไม่จำเป็นต้องเปลี่ยนแปลงค่าต่างๆ แต่ก็ยังมีเพิ่มข้อมูลที่เกี่ยวข้องกับการ compile และ ตัวcompile ที่สามารถปรับเปลี่ยนได้



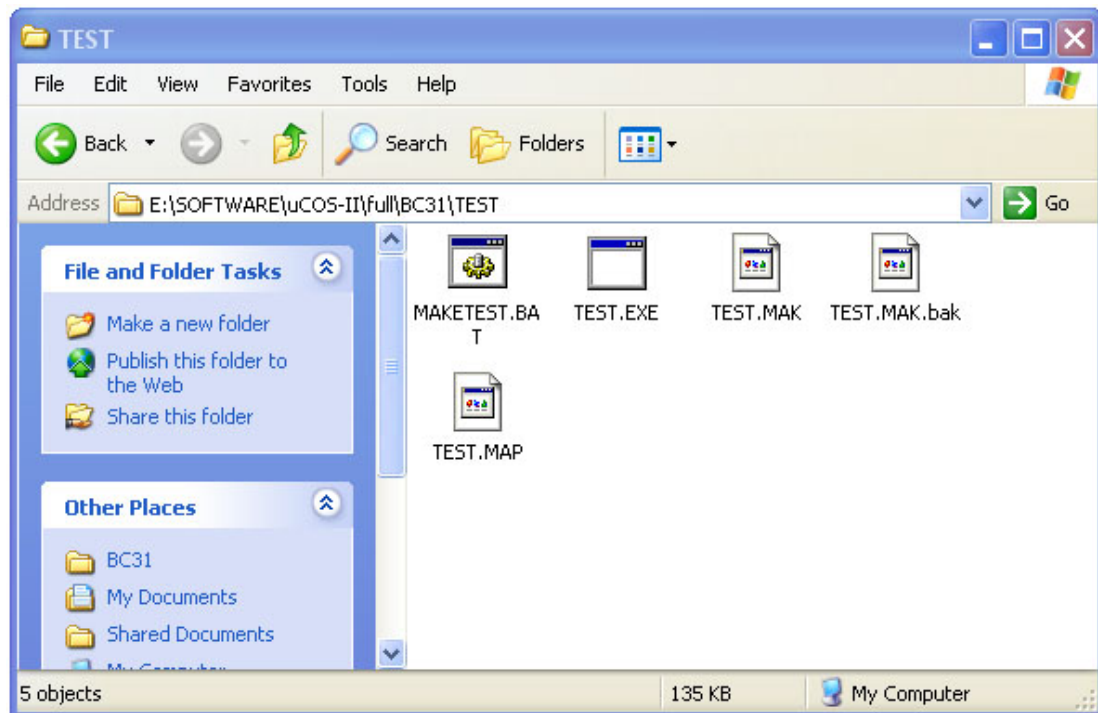
รูปที่ 36 เพิ่มข้อมูลส่วนโปรแกรมที่จะนำไป compile

โดยจะประกอบ เพิ่มข้อมูล ดังนี้

- เพิ่มข้อมูล devices.h และ devices.cpp เป็นเพิ่มข้อมูลที่ใช้ประกาศและแสดงลักษณะการทำงานของฟังก์ชันเมทอด ของชุดคำสั่งที่ใช้ควบคุมอุปกรณ์
- เพิ่มข้อมูล includes.h เป็นเพิ่มข้อมูลที่ประกาศเพิ่มข้อมูลที่เกี่ยวข้องกับการ compile ไมโครชิโอเอสทั้งหมด
- เพิ่มข้อมูล os_cfg.h เป็นเพิ่มข้อมูลที่กำหนดค่าการใช้งานฟังก์ชันและตัวแปรต่างๆ ของไมโครชิโอเอสว่าจะใช้หรือไม่ใช้อย่างไร
- เพิ่มข้อมูล test.lnk เป็นเพิ่มข้อมูลที่กำหนดเพิ่มข้อมูลออบเจกต์ที่จะเกี่ยวข้องในการ compile
- เพิ่มข้อมูล test.cpp เป็นเพิ่มข้อมูลโปรแกรมที่เขียนเป็นไมโครชิโอเอสแล้ว เตรียมนำไป compile ให้เป็นเพิ่มข้อมูล .exe ที่ทำงานได้

ส่วนที่กล่าวมาทั้งหมดนี้เป็นส่วนที่เกี่ยวข้องกับการใช้งานไมโครชิโอเอสและการ compile ไมโครชิโอเอส ที่สามารถปรับแต่งได้ตามที่ต้องการ แต่สำหรับการ compile ตัวที่ต้องปรับแต่งมากที่สุดคือ test.lnk ซึ่งจะกล่าวในส่วนถัดไป

นอกจากนี้ยังมีเพิ่มข้อมูลในส่วนที่ใช้ในการ compile อาจต้องจำเป็นในการปรับแต่งเพื่อให้สามารถ compile ได้

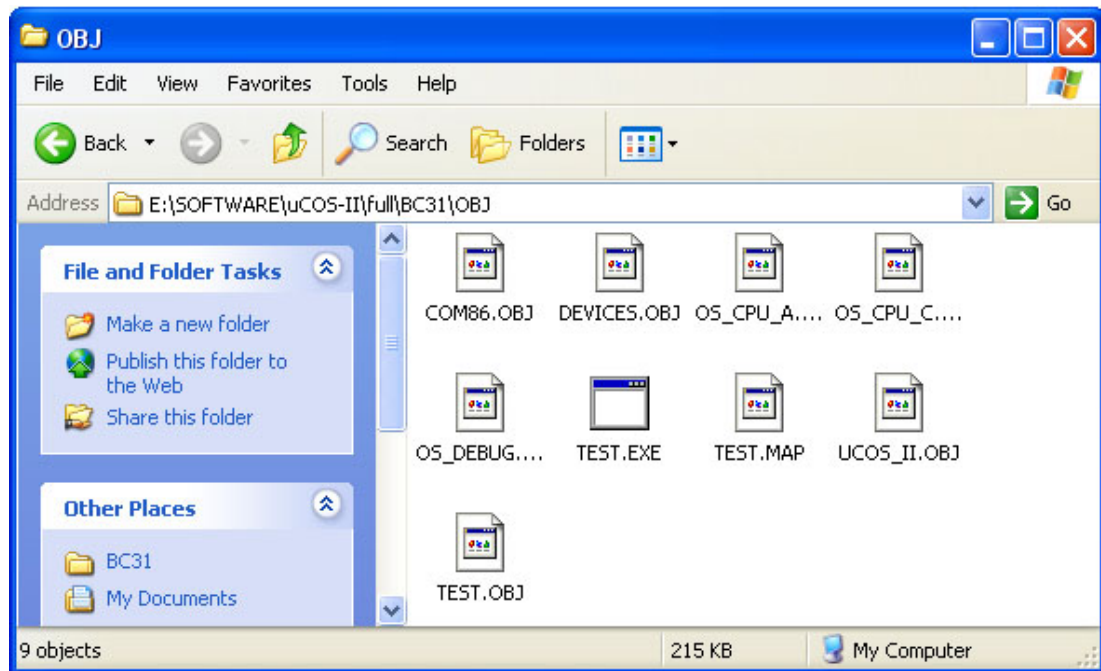


รูปที่ 37 แฟ้มข้อมูลส่วนโปรแกรมใช้สำหรับการ compile

โดยจะประกอบ แฟ้มข้อมูล ดังนี้

- แฟ้มข้อมูล test.mak เป็นแฟ้มข้อมูลประเภท makefile ที่ใช้ในการ compile ไมโครซีไอเอส 2 ซึ่งจะอธิบายในส่วนถัดไป
- แฟ้มข้อมูล test.map เป็นแฟ้มข้อมูลได้จากการ compile
- แฟ้มข้อมูล maketest.bat เป็นแฟ้มข้อมูลประเภท batfile ที่กำหนดค่าให้เรียก แฟ้มข้อมูล test.mak มาทำงาน
- แฟ้มข้อมูล test.exe เป็นแฟ้มข้อมูลได้จากการ compile ซึ่งพร้อมที่จะทำงาน

นอกจากนั้นยังมีแฟ้มข้อมูลต่างๆที่เกี่ยวข้องที่ได้จากการ compile ไมโครซีไอเอส 2



รูปที่ 37 แฟ้มข้อมูลส่วนโปรแกรมที่ได้จากการ compile

โดยจะประกอบด้วยโปรแกรมประเภท objectfile ของแฟ้มข้อมูลต่างๆที่เกี่ยวข้องกับแฟ้มข้อมูลที่จะนำไปใช้งาน

ลักษณะของชุดคำสั่งควบคุมอุปกรณ์และการทำงานของไมโครชีโอเอส

ชุดคำสั่งควบคุมอุปกรณ์สำหรับหุ่นยนต์แบ่งเป็น 2 ส่วน

1. ส่วนของแฟ้มข้อมูลแบบไลเบอรรี่ ที่เก็บคลาสของอุปกรณ์ชนิดต่างไว้
2. ส่วนของทาสก์ที่ใช้ควบคุมอุปกรณ์แต่ละชนิด

■ ส่วนของแฟ้มข้อมูลแบบไลเบอรรี่

จะประกาศคลาสต่างๆไว้ในแฟ้มข้อมูล devices.h และจะเก็บโปรแกรมแสดงการทำงานของเมททอดในแต่คลาสไว้ในแฟ้มข้อมูล devices.cpp เพื่อให้ผู้ใช้งานเรียกใช้ได้ง่าย

แฟ้มข้อมูล devices.h

```
//----- DC MOTOR -----  
  
class DCmotor  
{  
  
    private :  
  
        OS_EVENT *Mbox;  
        INT16U port_out1;  
        char port_status;  
  
    public :  
  
        char name[12];  
        void setDCmotor(OS_EVENT *M,INT16U p1,int p2,char p3,char N[]);  
        void forward();  
        void backward();  
        void stop();  
  
};
```

```
//----- STEP MOTOR -----
```

```
class STEPMotor  
{  
  
    private :  
  
        OS_EVENT *Mbox;
```

```

        int port_out;
        char port_status;
    public :
        char name[12];
        void setSTEPmotor(OS_EVENT *M,INT16U p1,char p3,char N[]);
        void forward ();
        void backward();
        void stop();
};

```

//----- IR SENSOR -----

```

class IRsensor
{
    private :
        OS_EVENT *Mbox;
        INT16U port_in;
    public :
        char name[12];
        void setIRsensor(OS_EVENT *M,INT16U p1,char N[]);
        int detect();
};

```

//----- BUMPED SENSOR -----

```

class BUMPsensor
{
    private :
        OS_EVENT *Mbox;
        INT16U port_in;
    public :
        char name[12];
        void setBUMPsensor(OS_EVENT *M,INT16U p1,char N[]);
};

```

```

        int bump();
    };

//----- End all Class of devices.h -----

```

จากเพิ่มข้อมูล devices.h จะแสดงค่าแอดทริบิวและเมทอด ของคลาสต่างๆ โดยจะเห็นว่าแต่ละคลาสจะประกอบด้วยส่วนหลักๆดังนี้

- แอดทริบิว Mbox เป็นตัวแปรชนิด พ้อยเตอร์ของ OS_EVENT ซึ่งจะมีไว้สำหรับอ้างถึงเมล์บ็อกที่ใช้ในการสื่อสารระหว่างอุปกรณ์ กับทาสก์ที่ควบคุมอุปกรณ์ชนิดนั้นๆ โดยอุปกรณ์ 1 ตัวจะมี 1 เมล์บ็อก
- แอดทริบิว port_in,port_out เป็นตัวแปรชนิด int ซึ่งจะใช้เก็บว่าเป็นอุปกรณ์ตัวใด ในจำนวนอุปกรณ์ที่เป็นชนิดเดียวกันทั้งหมด หรือก็คือการกำหนดหมายเลขให้กับอุปกรณ์เพื่อสามารถส่งงานอุปกรณ์ได้ตรงตามที่ต้องการ
- แอดทริบิว name เป็นตัวแปรชนิด อาร์เรย์ของ char ซึ่งจะใช้เก็บชื่อของอุปกรณ์ตามที่ผู้ใช้กำหนดไว้
- เมทอด ที่ขึ้นต้นด้วยคำว่า set...(...) จะเป็นเมทอดที่ใช้ในการกำหนดค่าแอดทริบิวต่างๆให้กับคลาส

ส่วนของเมทอดนอกเหนือจากนั้นจะเป็นเมทอดที่ใช้ในการสั่งงานควบคุมอุปกรณ์ของแต่ละชนิด

เพิ่มข้อมูล devices.cpp

```

#include "includes.h"

//----- DC MOTOR -----

void DCmotor ::setDCmotor(OS_EVENT *M,INT16U p1,char p3,char N[])
{
    Mbox = M;
    port_out1 = p1;
    port_status = p3;
    strcpy(name,N);
}

```

```

void DCMotor::forward()
{
    static char jms[3];

    port_status = jms[port_out1-1];

    jms[port_out1-1] = 0x01<<((port_out1-1)*2);

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out1-1]);

}

```

```

void DCMotor::backward()
{
    static char jms[3];

    port_status = jms[port_out1-1];

    jms[port_out1-1] = 0x02<<((port_out1-1)*2);

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out1-1]);

}

```

```

void DCMotor::stop()
{
    static char jms;

    port_status = jms;

    jms = 0;
}

```

```
OSMboxAccept(Mbox);
OSMboxPost(Mbox,(void *)&jms);
```

```
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุมดีซีมอเตอร์ โดยทั่วไปจะเป็นการคำนวณค่าของ ดีซีมอเตอร์ แต่ละตัวเพื่อส่งให้ทาสก์ดีซีมอเตอร์ทางเมล์บ็อก ประมวลผลส่งไปควบคุมดีซีมอเตอร์จริงๆ

```
//----- STEP MOTOR -----
```

```
void STEPmotor::setSTEPmotor(OS_EVENT *M,INT16U p1,char p3,char N[])
```

```
{
```

```
    char s[80];
```

```
    INT8U err;
```

```
    Mbox = M;
```

```
    port_out = p1;
```

```
    port_status = p3;
```

```
    strcpy(name,N);
```

```
}
```

```
void STEPmotor::forward()
```

```
{
```

```
    static char jms[2];
```

```
    port_status = jms[port_out-1];
```

```
    if (port_out==1)
```

```
    {
```

```
        jms[0]=12;
```

```
    }
```

```

else
{
    jms[1] =3;
}

OSMboxAccept(Mbox);
OSMboxPost(Mbox,(void *)&jms[port_out-1]);
}

```

```

void STEPmotor::backward()
{
    static char jms[2];

    port_status = jms[port_out-1];

    if (port_out==1)
    {
        jms[0] = 8;
    }
    else
    {
        jms[1] =2;
    }

    OSMboxAccept(Mbox);
    OSMboxPost(Mbox,(void *)&jms[port_out-1]);
}

```

```

void STEPmotor::stop()
{
    static char jms;

    port_status = jms;
    jms = 0;
}

```



```

    OSMboxAccept(Mbox);

    OSMboxPost(Mbox,(void *)&jms);

}

```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม สเตปมอเตอร์ โดยทั่วไปจะเป็นการคำนวณค่าของ สเตปมอเตอร์ แต่ละตัวในที่นี่ใช้ควบคุม สเตปมอเตอร์ 2 ตัวเพื่อส่งให้ทาสก์ สเตปมอเตอร์ผ่านทางเมล์บ็อก ประมวลผลส่งไปควบคุม สเตปมอเตอร์จริงๆ

```

//----- IR SENSOR -----

```

```

void IRsensor::setIRsensor(OS_EVENT *M,INT16U p1,char N[])
{
    Mbox = M;
    port_in = p1;
    strcpy(name,N);

}

```

```

int IRsensor::detect()
{
    char *IRin;

```

```

    IRin= (char *) OSMboxAccept(Mbox);
    OSMboxPost(Mbox,IRin);

```

```

        if( *IRin== 0x10 )
        {
            return 1;
        }
        else return 0;
    }

```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม อินฟราเรดเซ็นเซอร์ จะเป็นการรับข้อมูลมาจากเมล์บ็อกที่ส่งมาจากทาสก์ของ อินฟราเรดเซ็นเซอร์ โดยทาสก์อินฟราเรดเซ็นเซอร์ จะส่งค่ามาบอกว่า อินฟราเรดเซ็นเซอร์ตัวนั้น มีค่าเป็นอะไร (ทำงานอยู่หรือไม่ทำงาน) แล้วเมทอด detect() จะส่งค่ากลับไปให้ผู้เรียกว่า อินฟราเรดเซ็นเซอร์ตัวนั้น ทำงานอยู่หรือไม่ ถ้าทำงานจะส่งค่า 1 ถ้าไม่ทำงานจะส่งค่า 0

```
//----- BUMPED SENSOR -----
```

```
void BUMPSensor::setBUMPSensor(OS_EVENT *M,INT16U p1,char N[])
{
    Mbox = M;
    port_in = p1;
    strcpy(name,N);
}
```

```
int BUMPSensor::bump()
{
    char *Bump;

    Bump= (char *) OSMboxAccept(Mbox);
    OSMboxPost(Mbox,Bump);
    if( *Bump ==1)
        { return 1;}
    else { return 0;}
}
```

ในส่วนของการเพิ่มข้อมูล devices.cpp ที่ใช้ในการควบคุม บัมเซ็นเซอร์ จะเป็นการรับข้อมูลมาจากเมล์บ็อกที่ส่งมาจากทาสก์ของ บัมเซ็นเซอร์ โดยทาสก์อินฟราเรดเซ็นเซอร์ จะส่งค่ามาบอกว่า บัมเซ็นเซอร์ตัวนั้น มีค่าเป็นอะไร (ถูกกระทบหรือไม่ถูกกระทบ) แล้วเมทอด bump() จะส่งค่ากลับไปให้ผู้เรียกว่า บัมเซ็นเซอร์ตัวนั้น ถูกกระทบอยู่หรือไม่ ถ้าถูกกระทบจะส่งค่า 1 ถ้าไม่ถูกกระทบจะส่งค่า 0

ตัวอย่างวิธีการเรียกใช้ชุดคำสั่งที่เป็นไลบรารี

```
----- ส่วนของการประกาศออบเจ็ก -----
```

```
BUMPSensor B1;
```

```
STEPmotor step1;
```

```
STEPmotor step2;
```

----- ส่วนของการเรียกใช้งานเมทอด -----

```
if (B1.bump ())
```

```
{
```

```
    step1.backward ();
```

```
    step2.stop ();
```

```
    OSTimeDlyHMSM (0, 0, 1, 0);
```

```
    step1.forward ();
```

```
    OSTimeDlyHMSM (0, 0, 1, 0);
```

```
    step1.stop ();
```

```
}
```

```
else
```

```
{
```

```
    step1.stop ();
```

```
    step2.backward ();
```

```
    OSTimeDlyHMSM (0, 0, 1, 0);
```

```
    step2.forward ();
```

```
    OSTimeDlyHMSM (0, 0, 1, 0);
```

```
    step2.stop ();
```

```
}
```

* ส่วนการประกาศออบเจ็กต์ โดยปกติโปรแกรมช่วยเหลือจะประกาศให้ ผู้ใช้สามารถเรียกใช้ได้เลย

■ ส่วนของทาสก์ที่ใช้ควบคุมอุปกรณ์

ซึ่งจะแยกเก็บเป็นแฟ้มข้อมูลที่มีชื่อเหมือนอุปกรณ์นั้นๆ และจะถูกเรียกมารวมกับแฟ้มข้อมูลหลักเมื่อผู้ใช้เลือกอุปกรณ์ชนิดนั้น โดยจะมีหน้าที่ควบคุมอุปกรณ์นั้นๆ โดยตรง แบ่งเป็น

ทาสก์ดีชีมมอเตอร์

```
void Task2 (void *data)
{
    char *rxmsg;
    int sh,oldsh;
    data = data;

    for (;;) {
        oldsh=sh;
        rxmsg = (char *) OSMboxAccept (DCMbox);
        sh=*rxmsg;

        if ((sh != 116) &&( sh!= oldsh))
        {
            outportb(0x3020,sh);
        }

        OSTimeDlyHMSM (0, 0, 1, 0);           // Wait 1 second
    }
}
```

ทาสก์ดีชีมมอเตอร์จะรับข้อมูลมาจากเมล์บ็อกที่ชื่อ DCMbox แล้วนำมาตรวจสอบค่าก่อนส่งข้อมูลออกไปทาง ไอช่าบัสตำแหน่งที่ 3020 เพื่อควบคุมดีชีมมอเตอร์ จากโปรแกรมข้างบนยังควบคุมดีชีมมอเตอร์ 1 ตัวหากต้องการควบคุมเพิ่มจะต้องเพิ่ม เมล์บ็อกที่ส่งค่ามาให้สำหรับควบคุมดีชีมมอเตอร์ และเพิ่มส่วนประมวลผลจัดการสำหรับส่งข้อมูลออกไอช่าบัส โดยไอช่าบัส 1 ตำแหน่งควบคุมดีชีมมอเตอร์ได้

สูงสุด 4 ตัว ถ้าต้องการควบคุมดีซีมอเตอร์มากกว่านั้นต้องเพิ่มการอ้างตำแหน่งของไอซำบัส ส่วนของการ
หน่วงเวลาช่วงท้ายเป็นการกระทำเพื่อให้ทาสก์อื่นๆ สามารถแย่งชิงตัวประมวลผลไปทำงานได้ทำให้ไม่มี
ทาสก์ใดควบคุมตัวประมวลผลเพียงตัวเดียว

ทาสก์ สเตปมอเตอร์

```
void Task3 (void *data)
```

```
{  char *step1;
```

```
    char *step2;
```

```
    int sh,oldsh,sh1,sh2;
```

```
    sh=0;
```

```
    *step1=0;
```

```
    *step2=0;
```

```
    data = data;
```

```
    for (;;) {
```

```
        sh1=*step1;
```

```
        sh2=*step2;
```

```
        oldsh=sh;
```

```
        step1 = (char *) OSMboxAccept(Step1Mbox);
```

```
        OSMboxPost(Step1Mbox,step1);
```

```
        step2 = (char *)OSMboxAccept(Step2Mbox);
```

```
        OSMboxPost(Step2Mbox,step2);
```

```
        if (*step1==116)
```

```
            { *step1=sh1;}
```

```
        if (*step2==116)
```

```
            { *step2=sh2;}
```

```
        sh=*step1|*step2|0x10;
```

```
        if ((sh != 116) &&( sh!= oldsh))
```

```
            { outportb(0x3001,sh); }
```

```
        OSTimeDlyHMSM(0, 0, 0,70);
```

```
    }
```

```
}
```

ทาสก์ สเตปมอเตอร์จะรับข้อมูลจากเมล์บ็อกที่ชื่อ Step1Mbox และ Step2Mbox สำหรับควบคุม สเตปมอเตอร์ 2 ตัว แล้วนำมาประมวลผลรวมค่าที่ได้จากเมล์บ็อกทั้งสอง ส่งข้อมูลออกไปทางไอซำบัสตำแหน่งที่ 3001 เพื่อควบคุม สเตปมอเตอร์ จากโปรแกรมข้างบนยังควบคุม สเตปมอเตอร์ 2 ตัว หากต้องการควบคุมเพิ่มจะต้องเพิ่ม เมล์บ็อกที่ส่งค่ามาให้สำหรับควบคุม สเตปมอเตอร์และเพิ่มส่วนประมวลผลจัดการสำหรับส่งข้อมูลออกไอซำบัส โดยไอซำบัส 1 ตำแหน่งควบคุม สเตปมอเตอร์ได้สูงสุด 3 ตัว (สามารถดัดแปลงให้ควบคุมถึง 4 ตัวได้) ถ้าต้องการควบคุม สเตปมอเตอร์มากกว่านั้นต้องเพิ่มการอ้างตำแหน่งของไอซำบัส

ทาสก์อินฟารเคชั่นเซอร์

```
void Task4 (void *data)
{
    INT16U pt;
    char inIR,inIR1;
    char true,false;

    data = data;
    true=0x10;
    false=0x11;

    for (;;) {
        pt=0x3004;
        inIR = inportb(pt);

        inIR1=inIR&(0x01<<0);
        if (inIR1==0x01)
        {
            OSMboxAccept(IR1Mbox);
            OSMboxPost(IR1Mbox, (void *)true);
        }
        else
        {
            OSMboxAccept(IR1Mbox);
            OSMboxPost(IR1Mbox, (void *)false);
        }
    }
}
```

```
}
```

```
inIR1=inIR&(0x01<<1);  
if (inIR1==0x02)  
{  
    OSMboxAccept(IR2Mbox);  
    OSMboxPost(IR2Mbox, (void *)true);  
}  
else  
{  
    OSMboxAccept(IR2Mbox);  
    OSMboxPost(IR2Mbox, (void *)false);  
}
```

```
inIR1=inIR&(0x01<<2);  
if (inIR1==0x04)  
{  
    OSMboxAccept(IR3Mbox);  
    OSMboxPost(IR3Mbox, (void *)true);  
}  
else  
{  
    OSMboxAccept(IR3Mbox);  
    OSMboxPost(IR3Mbox, (void *)false);  
}
```

```
inIR1=inIR&(0x01<<3);  
if (inIR1==0x08)  
{  
    OSMboxAccept(IR4Mbox);
```

```
    OSMboxPost(IR4Mbox, (void *)true);  
}  
else  
{  
    OSMboxAccept(IR4Mbox);  
    OSMboxPost(IR4Mbox, (void *)false);  
}
```

```
inIR1=inIR&(0x01<<4);  
if (inIR1==0x010)  
{  
    OSMboxAccept(IR5Mbox);  
    OSMboxPost(IR5Mbox, (void *)true);  
}  
else  
{  
    OSMboxAccept(IR5Mbox);  
    OSMboxPost(IR5Mbox, (void *)false);  
}
```

```
inIR1=inIR&(0x01<<5);  
if (inIR1==0x20)  
{  
    OSMboxAccept(IR6Mbox);  
    OSMboxPost(IR6Mbox, (void *)true);  
}  
else  
{  
    OSMboxAccept(IR6Mbox);  
    OSMboxPost(IR6Mbox, (void *)false);  
}
```



```

inIR1=inIR&(0x01<<6);
if (inIR1==0x40)
{
    OSMboxAccept(IR7Mbox);
    OSMboxPost(IR7Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR7Mbox);
    OSMboxPost(IR7Mbox, (void *)false);
}

inIR1=inIR&(0x01<<7);
if (inIR1==0x80)
{
    OSMboxAccept(IR8Mbox);
    OSMboxPost(IR8Mbox, (void *)true);
}
else
{
    OSMboxAccept(IR8Mbox);
    OSMboxPost(IR8Mbox, (void *)false);
}

OSTimeDlyHMSM(0, 0, 0,30);
}
}

```

ทาสก์อินฟารเคชั่นเซอร์จะรับข้อมูลมาทางตำแหน่งที่ 3004 ของไอช่าบัสแล้วนำมาประมวลผลว่าอินฟารเคชั่นเซอร์แต่ละตัวมีค่าเป็นอะไรแล้วส่งค่าไปเก็บไว้ในเมล์บ็อกของอินฟารเคชั่นเซอร์แต่ละตัวเพื่อให้ผู้ใช้งานสามารถเรียกค่าของอินฟารเคชั่นเซอร์แต่ละตัวมาใช้งานได้ทันที โดยโปรแกรมข้างบนจะจัดการกับอินฟารเคชั่นเซอร์จำนวน 8 ตัว ถ้าใช้น้อยกว่าจำนวนที่กำหนดให้สามารถแก้ไขลบส่วนที่ไม่ใช่ออกไปได้ แต่ถ้าต้องการเพิ่มจำนวนต้องมีการเพิ่มการอ้างตำแหน่งของไอช่าบัสในการรับค่าเนื่องจาก ไอช่าบัส 1 ตำแหน่งควบคุมอินฟารเคชั่นเซอร์ได้จำนวน 8 ตัว

ทาสก์บั้มเซ็นเซอร์

```
void Task5 (void *data)
```

```
{
```

```
    char true,false;
```

```
    INT8U inBump,tempbump,tem;
```

```
    data = data;
```

```
    true=0x01;
```

```
    false=0x11;
```

```
    for (;;) {
```

```
        inBump = inportb(0x3010);
```

```
        tempbump=inBump|0xfe;
```

```
        if (tempbump==0xfe)
```

```
        {
```

```
            OSMboxAccept(Bump1Mbox);
```

```
            OSMboxPost(Bump1Mbox, (void *)&true);
```

```
        }
```

```
        else
```

```
        {
```

```
OSMboxAccept(Bump1Mbox);
OSMboxPost(Bump1Mbox, (void *)&false);
}
```

```
tempbump=inBump|0xfd;
if (tempbump ==0xfd)
{
OSMboxAccept(Bump2Mbox);
OSMboxPost(Bump2Mbox, (void *)&true);
}
else
{
OSMboxAccept(Bump2Mbox);
OSMboxPost(Bump2Mbox, (void *)&false);
}
```

```
tempbump=inBump|0xfb;
if (tempbump==0xfb)
{
OSMboxAccept(Bump3Mbox);
OSMboxPost(Bump3Mbox, (void *)&true);
}
else
{
OSMboxAccept(Bump3Mbox);
OSMboxPost(Bump3Mbox, (void *)&false);
}
```

```
tempbump=inBump|0xf7;
if (tempbump==0xf7)
{
```

```
OSMboxAccept(Bump4Mbox);
OSMboxPost(Bump4Mbox, (void *)&true);
}

else
{
    OSMboxAccept(Bump4Mbox);
    OSMboxPost(Bump4Mbox, (void *)&false);
}
```

```
tempbump=inBump|0xef;
if (tempbump==0xef)
{
    OSMboxAccept(Bump5Mbox);
    OSMboxPost(Bump5Mbox, (void *)&true);
}

else
{
    OSMboxAccept(Bump5Mbox);
    OSMboxPost(Bump5Mbox, (void *)&false);
}
```

```
tempbump=inBump|0xdf;
if (tempbump==0xdf)
{
    tem=1;
    OSMboxAccept(Bump6Mbox);
    OSMboxPost(Bump6Mbox, (void *)&true);
}

else
{
    OSMboxAccept(Bump6Mbox);
    OSMboxPost(Bump6Mbox, (void *)&false);
}
```

```

tempbump=inBump|0xbf;
if (tempbump==0xbf)
{
    OSMboxAccept(Bump7Mbox);
    OSMboxPost(Bump7Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump7Mbox);
    OSMboxPost(Bump7Mbox, (void *)&false);
}

tempbump=inBump|0x7f;
if (tempbump==0x7f)
{
    OSMboxAccept(Bump8Mbox);
    OSMboxPost(Bump8Mbox, (void *)&true);
}
else
{
    OSMboxAccept(Bump8Mbox);
    OSMboxPost(Bump8Mbox, (void *)&false);
}

OSTimeDlyHMSM(0, 0,1,0);
}
}

```

ทาสก์บัมเซ็นเซอร์จะทำงานคล้ายกับ ทาสก์อินฟาเรดเซ็นเซอร์ แต่ต่างกันที่ ทาสก์บัมเซ็นเซอร์ทำงานที่ค่าเป็น 0 และค่าปกติเป็น 1

โดยทาสก์ต่างจะถูกรวมไว้ภายใต้โครงสร้างการทำงานของไมโครชิโอเอส2 ซึ่งมีรูปแบบดังนี้

```
#include "includes.h"
```

```
#define TASK_STK_SIZE 512 // กำหนด ขนาด สแต็ก ของทาสก์
```

```
////////// กำหนด ID //////////
```

```
#define TASK_START_ID 0 /* Application tasks IDs */
```

```
#define TASK_CLK_ID 1
```

```
#define TASK_1_ID 2
```

```
#define TASK_2_ID 3
```

```
#define TASK_3_ID 4
```

```
#define TASK_4_ID 5
```

```
#define TASK_5_ID 6
```

```
////////// กำหนด Priority //////////
```

```
#define TASK_START_PRIO 10 /* Application tasks priorities */
```

```
#define TASK_CLK_PRIO 17
```

```
#define TASK_1_PRIO 12
```

```
#define TASK_2_PRIO 13
```

```
#define TASK_3_PRIO 14
```

```
#define TASK_4_PRIO 15
```

```
#define TASK_5_PRIO 16
```

```
////////// กำหนด task stack //////////
```

```
OS_STK TaskStartStk[TASK_STK_SIZE]; /* Startup task stack */
```

```
OS_STK TaskClkStk[TASK_STK_SIZE]; /* Clock task stack */
```

```
OS_STK Task1Stk[TASK_STK_SIZE]; /* Task #1 task stack */
```

```
OS_STK Task2Stk[TASK_STK_SIZE]; /* Task #2 task stack */
```

```
OS_STK Task3Stk[TASK_STK_SIZE]; /* Task #3 task stack */
```

```
OS_STK Task4Stk[TASK_STK_SIZE]; /* Task #4 task stack */
```

```
OS_STK Task5Stk[TASK_STK_SIZE]; /* Task #5 task stack */
```

```
////////// กำหนด task //////////
```

```
void TaskStart(void *data); /* Function prototypes of tasks */
```

```

static void TaskStartCreateTasks(void);
static void TaskStartDispInit(void);
static void TaskStartDispInit2(int);
static void TaskStartDisp(void);
        void TaskClk(void *data);
        void Task1(void *data);
        void Task2(void *data);
        void Task3(void *data);
        void Task4(void *data);
        void Task5(void *data);

OS_EVENT  *DispSem  /// SEMAPHORES สำหรับแสดงผล ///
/// create os event for mailbox for device //////////////////////////////////

OS_EVENT  *DCMbox;
OS_EVENT  *Step1Mbox;
OS_EVENT  *Step2Mbox;
OS_EVENT  *IR1Mbox;
OS_EVENT  *IR2Mbox;
OS_EVENT  *IR3Mbox;
OS_EVENT  *Bump1Mbox;
OS_EVENT  *Bump2Mbox;
OS_EVENT  *Bump3Mbox;

OS_EVENT  *IR4Mbox;
OS_EVENT  *IR5Mbox;
OS_EVENT  *IR6Mbox;
OS_EVENT  *IR7Mbox;
OS_EVENT  *IR8Mbox;

OS_EVENT  *Bump4Mbox;
OS_EVENT  *Bump5Mbox;
OS_EVENT  *Bump6Mbox;
OS_EVENT  *Bump7Mbox;

```

```

OS_EVENT  *Bump8Mbox;
OS_EVENT  *SonarMbox;
OS_EVENT  *Senddis1;
OS_EVENT  *Senddis2;

////////// จป create os event for mailbox for device//////////

////////// ประกาศ object ตามชื่อ : ชื่อคลาส ชื่อออปเจก(ที่กำหนด) //////////

//// ตัวอย่าง ////

BUMPsensor B1;
BUMPsensor B2;
STEPmotor step1;
STEPmotor step2;

////////// จป ประกาศ object//////////

void main (void) ////////// main uCOS //////////
{
    OS_STK *ptos;
    OS_STK *pbos;
    INT32U size;

    COM86_DispInit(); ///กำหนดค่าเริ่มต้นของหน้าจอแสดงผล////
    COM86_DispClrScr(DISP_BLACK, DISP_WHITE);          /* Clear the screen */

    OSInit();                      /* Initialize uC/OS-II */
    COM86_DOSSaveReturn();          /* Save environment to return to DOS */
    COM86_VectSet(uCOS, OSCtxSw);   /* Install uC/OS-II's context switch vector */
    DispSem  = OSSemCreate(1);      /* Console display semaphore */
    COM86_ElapsedInit();            /* Initialized elapsed time measurement */

    ptos      = &TaskStartStk[TASK_STK_SIZE - 1];    /* TaskStart() will use Floating-Point */

```



```

pbos    = &TaskStartStk[0];
size    = TASK_STK_SIZE;

OSTaskStkInit_FPE_x86(&ptos, &pbos, &size);

,

////////// ຕັ້ງໜ້າ taskstart //////////

OSTaskCreateExt(TaskStart,

    (void *)0,

    ptos,

    TASK_START_PRIO,

    TASK_START_ID,

    pbos,

    size,

    (void *)0,

    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

OSStart();                                /* Start multitasking */

////////// ຈົບ ຕັ້ງໜ້າ taskstart //////////

}

/*

*****

*****

*                               STARTUP TASK

*****

*****

*/

void TaskStart(void *pdata)

{

    #if OS_CRITICAL_METHOD == 3                /* Allocate storage for CPU status register */

        OS_CPU_SR cpu_sr;

    #endif

    char s[80];

```

```

    INT16S    key;

    INT8U    err;

    pdata = pdata;                                /* Prevent compiler warning */

TaskStartDispInit();                            /* Setup the display */

    OS_ENTER_CRITICAL();                        /* Install uC/OS-II's clock tick ISR */
    COM86_VectSet(0x08, OSTickISR);
    COM86_SetTickRate(OS_TICKS_PER_SEC);        /* Reprogram tick rate */
    OS_EXIT_CRITICAL();

    OSStatInit();                              /* Initialize uC/OS-II's statistics */

//////////////////////////////// create mail box //////////////////////////////////
/* Create message mailboxes */

    DCMbox = OSMboxCreate((void *)0);
    Step1Mbox = OSMboxCreate((void *)0);
    Step2Mbox = OSMboxCreate((void *)0);
    IR1Mbox = OSMboxCreate((void *)0);
    IR2Mbox = OSMboxCreate((void *)0);
    IR3Mbox = OSMboxCreate((void *)0);
    Bump1Mbox = OSMboxCreate((void *)0);
    Bump2Mbox = OSMboxCreate((void *)0);
    Bump3Mbox = OSMboxCreate((void *)0);

    IR4Mbox = OSMboxCreate((void *)0);
    IR5Mbox = OSMboxCreate((void *)0);
    IR6Mbox = OSMboxCreate((void *)0);
    IR7Mbox = OSMboxCreate((void *)0);
    IR8Mbox = OSMboxCreate((void *)0);

```

```

Bump4Mbox = OSMboxCreate((void *)0);
Bump5Mbox = OSMboxCreate((void *)0);
Bump6Mbox = OSMboxCreate((void *)0);
Bump7Mbox = OSMboxCreate((void *)0);
Bump8Mbox = OSMboxCreate((void *)0);

```

```

Senddis1= OSMboxCreate((void *)0);
Senddis2= OSMboxCreate((void *)0);
SonarMbox = OSMboxCreate((void *)0);

```

```

////////// call init variable //////////

```

```

//////// เรียกเมททอดในการ กำหนดค่าเริ่มต้นของ แอดทริบิว //////////

```

```

step1.setSTEPmotor(Step1Mbox,1,0,"jo");
step2.setSTEPmotor(Step2Mbox,2,0,"jojo");
B1.setBUMPsensor(Bump1Mbox,1,"jo1");
B2.setBUMPsensor(Bump2Mbox,2,"jo2");

```

```

//////////จบ กำหนดค่าเริ่มต้น //////////

```

```

TaskStartCreateTasks();                /* Create all other tasks          */
    for (;;) {
        TaskStartDisp();                /* Update the display              */
        if (COM86_GetKey(&key)) {        /* See if key has been pressed     */
            if (key == 0x1B) {            /* Yes, see if it's the ESCAPE key */
                COM86_DOSReturn();        /* Yes, return to DOS              */
            }
        }
        OSCtxSwCtr = 0;                  /* Clear context switch counter    */
        OSTimeDly(OS_TICKS_PER_SEC);     /* Wait one second                  */
    }
}

////////// จบ task main uCOS-
II//////////

```

```

//////////////////////////////// task กำหนดค่าเริ่มต้นในการแสดงผล //////////////////////////////////
static void TaskStartDispInit (void)
{
    COM86_DispStr( 1, 1, " Robot Mornitor ",
DISP_WHITE + DISP_BLINK, DISP_RED);
    COM86_DispStr( 1, 2, "
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 3, " Device ",
DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 4, " Stepmotor ",
DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 5, " Stepmotor ",
DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 6, " IRsensor (IR8-IR0)
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 7, " Bumpsensor (bump8-bump0)
", DISP_BLACK, DISP_WHITE);
    COM86_DispStr( 1, 23, " <-PRESS 'ESC' TO QUIT-> ",
DISP_BLACK + DISP_BLINK, DISP_WHITE);
}

//////////////////////////////// จบ task กำหนดค่าเริ่มต้นในการแสดงผล //////////////////////////////////
//////////////////////////////// task กำหนดค่าการแสดงผล //////////////////////////////////
static void TaskStartDisp (void)
{
    char s[80];
    INT8U err;

    char *temp;
    char temp1;

    temp = (char *)OSMboxAccept(Step1Mbox);
    OSMboxPost(Step1Mbox, temp);

    if (*temp ==12)

```

```

{
    sprintf(s, "%s\tforward ",step1.name);
        OSSemPend(DispSem, 0, &err);
        COM86_Dispatch(25, 4, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp ==8)
{
    sprintf(s, "%s\tbackward",step1.name);
        OSSemPend(DispSem, 0, &err);
        COM86_Dispatch(25, 4, s, DISP_YELLOW, DISP_BLUE);
    OSSemPost(DispSem);
}

else if (*temp ==0)
    {
        sprintf(s, "%s\tstop  ",step1.name);
        OSSemPend(DispSem, 0, &err);
        COM86_Dispatch(25, 4, s, DISP_YELLOW, DISP_BLUE);
        OSSemPost(DispSem);
    }

    temp = (char *)OSMboxAccept(Step2Mbox);
    OSMboxPost(Step2Mbox, temp);

    if (*temp ==3)
{
    sprintf(s, "%s\tforward ",step2.name);
        OSSemPend(DispSem, 0, &err);
        COM86_Dispatch(25, 5, s, DISP_YELLOW, DISP_BLUE);
        OSSemPost(DispSem);
}

else if (*temp ==2)
{
    sprintf(s, "%s\tbackward",step2.name);

```

```

        OSSemPend(DispSem, 0, &err);

        COM86_DispatchStr(25, 5, s, DISP_YELLOW, DISP_BLUE);

        OSSemPost(DispSem);

    }

    else if (*temp == 0)
    {
        sprintf(s, "%s\stop ", step2.name);

        OSSemPend(DispSem, 0, &err);

        COM86_DispatchStr(25, 5, s, DISP_YELLOW, DISP_BLUE);

        OSSemPost(DispSem);

    }

    temp1 = inportb(0x3004);

    sprintf(s, "\t%x", temp1);

    OSSemPend(DispSem, 0, &err);

    COM86_DispatchStr(25, 6, s, DISP_YELLOW, DISP_BLUE);

    OSSemPost(DispSem);

    sprintf(s, "\t ");

    OSSemPend(DispSem, 0, &err);

    COM86_DispatchStr(27, 6, s, DISP_YELLOW, DISP_BLUE);

    OSSemPost(DispSem);

    temp1 = inportb(0x3010);

    sprintf(s, "\t%x ", temp1);

    OSSemPend(DispSem, 0, &err);

    COM86_DispatchStr(25, 7, s, DISP_YELLOW, DISP_BLUE);

    OSSemPost(DispSem);

    sprintf(s, "\t ");

    OSSemPend(DispSem, 0, &err);

    COM86_DispatchStr(27, 7, s, DISP_YELLOW, DISP_BLUE);

    OSSemPost(DispSem);

}

```

```

//////////////////////////////// จป task กำหนดค่าการแสดงผล //////////////////////////////////
/*
*****

*****

*
CREATE TASKS

*****

*****

*/

//////////////////////////////// มีไว้สร้าง task //////////////////////////////////

static void TaskStartCreateTasks (void)
{

//////////////////////////////// สร้างไว้ตามจำนวนที่ ประกาศไว้ข้างบน
////////////////////////////////

    OSTaskCreateExt(TaskClk, /// ชื่อ////////////////////////////////
        (void *)0, /// ตามมัน////////////////////////////////
        &TaskClkStk[TASK_STK_SIZE - 1], /// stack ตาม top //////////////////////////////////
        TASK_CLK_PRIO, /// task prio //////////////////////////////////
        TASK_CLK_ID, /// task id //////////////////////////////////
        &TaskClkStk[0], /// stack ตาม bottum //////////////////////////////////
        TASK_STK_SIZE, /// fix //////////////////////////////////
        (void *)0, /// fix //////////////////////////////////
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR); ////////////////////////////////// fix
    //////////////////////////////////

    OSTaskCreateExt(Task1,
        (void *)0,
        &Task1Stk[TASK_STK_SIZE - 1],
        TASK_1_PRIO,
        TASK_1_ID,
        &Task1Stk[0],
        TASK_STK_SIZE,
        (void *)0,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

```

```
OSTaskCreateExt(Task2,  
    (void *)0,  
    &Task2Stk[TASK_STK_SIZE - 1],  
    TASK_2_PRIO,  
    TASK_2_ID,  
    &Task2Stk[0],  
    TASK_STK_SIZE,  
    (void *)0,  
    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
```

```
OSTaskCreateExt(Task3,  
    (void *)0,  
    &Task3Stk[TASK_STK_SIZE - 1],  
    TASK_3_PRIO,  
    TASK_3_ID,  
    &Task3Stk[0],  
    TASK_STK_SIZE,  
    (void *)0,  
    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
```

```
OSTaskCreateExt(Task4,  
    (void *)0,  
    &Task4Stk[TASK_STK_SIZE-1],  
    TASK_4_PRIO,  
    TASK_4_ID,  
    &Task4Stk[0],  
    TASK_STK_SIZE,  
    (void *)0,  
    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
```

```
OSTaskCreateExt(Task5,  
    (void *)0,  
    &Task5Stk[TASK_STK_SIZE-1],
```



```

TASK_5_PRIO,
TASK_5_ID,
&Task5Stk[0],
TASK_STK_SIZE,
(void *)0,
OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
}

//////////////////////////////////จบ มีไว้สร้าง task //////////////////////////////////

////////////////////////////////// main user //////////////////////////////////

void Task1 (void *pdata)
{
////////////////////////////////// ผู้ใช้กำหนดตัวแปล //////////////////////////////////

    pdata=pdata;

////////////////////////////////// ผู้ใช้กำหนด โปรแกรม //////////////////////////////////

    for (;;)
    {

    }

}

////////////////////////////////// จบ main user //////////////////////////////////

////////////////////////////////// ส่วนของโปรแกรมของ task ต่างๆที่กล่าวมาข้างต้น //////////////////////////////////

ทาสก์ดีชีมมเตอร์
ทาสก์ สเตปมมเตอร์
ทาสก์อินฟารเคชั่นเซอร์
ทาสก์บัมชั่นเซอร์

////////////////////////////////// จบ task ต่างๆที่กล่าวมาข้างต้น //////////////////////////////////

***** จบโครงสร้าง uCOS-II *****

///*****///

```

แฟ้มข้อมูล includes.h

```
#ifndef      _INCLUDE_H_
#define      _INCLUDE_H_

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
#include <conio.h>
#include <dos.h>
#include <setjmp.h>

#ifdef __cplusplus
extern "C" {
#endif

#include "software\ucos-ii\c86l\bc31\os_cpu.h"          ///// * ส่วนที่เป็นภาษา C
#include "os_cfg.h"
#include "software\ucos-ii\source\ucos_ii.h"
#include "software\blocks\com86\bc31\am186cc.h"
#include "software\blocks\com86\bc31\com86.h"

#ifdef __cplusplus
#include "devices.h"                                     ///// * ส่วนที่เป็นภาษา C++
}
#endif
#endif
```

ส่วนสำคัญของแฟ้มข้อมูล includes.h ส่วนที่ใช้กำหนดให้มีการ compile ร่วมกันระหว่าง ภาษา C และ ภาษา C++ โดยสามารถกำหนดการวางตำแหน่ง ส่วนต่างๆเหมือนตัวอย่างข้างต้น

การสร้างและการปรับแต่งแฟ้มข้อมูลสำหรับการคอมไพล์แฟ้มข้อมูลที่ใช้งานไมโครซีโอเอส2

ในการคอมไพล์จะต้องมีการสร้างแฟ้มข้อมูล ขึ้นมาเพื่อเรียกใช้งานโปรแกรมที่ใช้งานซึ่งภายในแฟ้มข้อมูลที่สร้างขึ้นมาจะประกอบด้วย (ใช้แฟ้มข้อมูลนามสกุล .MAK)

1. การกำหนดเครื่องมือที่ใช้มาในการคอมไพล์

BORLAND=c:\borlandc กำหนดที่อยู่ของโปรแกรม Borlandc

CC=\$(BORLAND)\BIN\BCC กำหนด path ต่างๆ

ASM=\$(BORLAND)\BIN\TASM

LINK=\$(BORLAND)\BIN\TLINK

TOUCH=\$(BORLAND)\BIN\TOUCH

2. ใส่ตำแหน่งไดเรกทอรีที่จะไปคอมไพล์

LST=..\LST กำหนด path ต่างๆ

OBJ=..\OBJ

SOURCE=..\SOURCE

TARGET=..\TEST

WORK=..\WORK

OS=\SOFTWARE\uCOS-II\SOURCE

COM86=\SOFTWARE\BLOCKS\COM86\BC31

PORT=\SOFTWARE\uCOS-II\C86L\BC31

3. การกำหนดค่าการคอมไพล์

#####

COMPILER FLAGS

#

-l Generate 80186 code

-B Compile and call assembler

-c Compiler to .OBJ

-G Select code for speed

-I Path to include directory

```

# -k-      Standard stack frame
# -L       Path to libraries directory
# -ml      Large memory model
# -N-      Do not check for stack overflow
# -n       Path to object directory
# -O       Optimize jumps
# -Ob      Dead code elimination
# -Oe      Global register allocation
# -Og      Optimize globally
# -Ol      Loop optimization
# -Om      Invariant code motion
# -Op      Copy propagation
# -Ov      Induction variable
# -v       Source debugging ON
# -vi      Turn inline expansion ON
# -wpro     Error reporting: call to functions with no prototype
# -Z       Suppress redundant loads

```

```
#####
```

ภาษาซี

```

C_FLAGS=-c -ml -l -G -O -Ogemvlp -Z -d -n..obj -k- -v -vi- -wpro -I$(BORLAND)\INCLUDE -
L$(BORLAND)\LIB

```

```
#####
```

```
#          ASSEMBLER FLAGS
```

```
#
```

```
# /MX      Case sensitive on globals
```

```
# /ZI      Full debug info
```

```
# /O       Generate overlay code
```

```
#####
```

ภาษาแอสเซมบลี

```
ASM_FLAGS=/MX /ZI /O
```

4.กำหนดเพิ่มข้อมูลเฮดเดอร์ที่เกี่ยวข้อง

```
INCLUDES= $(SOURCE)\INCLUDES.H \  
           $(SOURCE)\devices.H \    * headerfile ของชุดคำสั่งที่เพิ่ม  
           $(SOURCE)\OS_CFG.H \  
           $(PORT)\OS_CPU.H \  
           $(COM86)\COM86.H \  
           $(COM86)\AM186CC.H \  
           $(OS)\uCOS_II.H
```

* สามารถเพิ่มเติมส่วนที่เราสร้างขึ้นเองมาได้ ด้วยวิธีข้างต้น

5. กำหนดเพิ่มข้อมูลที่ใช้ในการสร้างเพิ่มข้อมูลที่จะนำไปใช้งาน

```
$(TARGET)\TEST.EXE: \  
    $(WORK)\INCLUDES.H \  
    $(OBJ)\OS_CPU_A.OBJ \  
    $(OBJ)\OS_CPU_C.OBJ \  
    $(OBJ)\OS_DEBUG.OBJ \  
    $(OBJ)\COM86.OBJ \  
    $(OBJ)\TEST.OBJ \  
    $(OBJ)\devices.OBJ \    * objectfile ของชุดคำสั่งที่เพิ่ม  
    $(OBJ)\uCOS_II.OBJ \  
    $(SOURCE)\TEST.LNK  
    COPY $(SOURCE)\TEST.LNK  
    $(LINK) $(LINK_FLAGS) @TEST.LNK  
    COPY $(OBJ)\TEST.EXE $(TARGET)\TEST.EXE  
    COPY $(OBJ)\TEST.MAP $(TARGET)\TEST.MAP  
    DEL TEST.MAK
```

* สามารถเพิ่มเติมส่วนที่เราสร้างขึ้นเองมาได้ ด้วยวิธีข้างต้น

6.ทำการสร้างเพิ่มข้อมูลที่จะนำมาประกอบเป็นเพิ่มข้อมูลทำงานได้

\$(WORK)\INCLUDES.H: \

\$(INCLUDES)

COPY \$(SOURCE)\INCLUDES.H INCLUDES.H

COPY \$(SOURCE)\devices.H devices.H

COPY \$(SOURCE)\OS_CFG.H OS_CFG.H

COPY \$(COM86)\COM86.H COM86.H

COPY \$(PORT)\OS_CPU.H OS_CPU.H

COPY \$(OS)\uCOS_II.H uCOS_II.H

\$(OBJ)\OS_CPU_A.OBJ: \

\$(PORT)\OS_CPU_A.ASM

COPY \$(PORT)\OS_CPU_A.ASM OS_CPU_A.ASM

\$(ASM) \$(ASM_FLAGS) \$(PORT)\OS_CPU_A.ASM,\$(OBJ)\OS_CPU_A.OBJ

\$(OBJ)\OS_CPU_C.OBJ: \

\$(PORT)\OS_CPU_C.C \

COPY \$(PORT)\OS_CPU_C.C OS_CPU_C.C

\$(CC) \$(C_FLAGS) OS_CPU_C.C

\$(OBJ)\OS_DEBUG.OBJ: \

\$(PORT)\OS_DEBUG.C \

COPY \$(PORT)\OS_DEBUG.C OS_DEBUG.C

\$(CC) \$(C_FLAGS) OS_DEBUG.C

\$(OBJ)\COM86.OBJ: \

\$(COM86)\COM86.C \

\$(INCLUDES)

COPY \$(COM86)\COM86.C COM86.C

\$(CC) \$(C_FLAGS) COM86.C

\$(OBJ)\devices.OBJ: \ *เพิ่มข้อมูลการทำงานของชุดคำสั่ง

\$(SOURCE)\devices.CPP \

\$(INCLUDES)

COPY \$(SOURCE)\devices.CPP devices.CPP

\$(CC) \$(C_FLAGS) devices.CPP

```
$(OBJ)\TEST.OBJ:      \ * เพิ่มข้อมูลโปรแกรม
$(SOURCE)\TEST.CPP    \
$(INCLUDES)
COPY $(SOURCE)\TEST.CPP TEST.CPP
$(CC) $(C_FLAGS)      TEST.CPP
```

```
$(OBJ)\uCOS_II.OBJ:    \
$(OS)\uCOS_II.C       \
$(INCLUDES)
COPY $(OS)\uCOS_II.C  uCOS_II.C
$(CC) $(C_FLAGS)      uCOS_II.C
```

* สามารถเพิ่มเติมส่วนที่เราสร้างขึ้นเองมาได้ และสามารถแก้ไขชื่อตามเพิ่มข้อมูลของเราได้

หลังจากสร้างเพิ่มข้อมูลในการสร้างเพิ่มข้อมูลที่ใช้งานแล้วต้องสร้างเพิ่มข้อมูลที่เก็บส่วนที่เกี่ยวข้องกับเพิ่มข้อมูลที่ทำงานด้วย (test .LNK) ซึ่งจะกำหนดไว้ในส่วนการกำหนดเพิ่มข้อมูลที่ใช้ในการสร้างเพิ่มข้อมูลที่จะนำไปใช้งาน

```
/v /s /c /P-      +
c:\borlandc\LIB\C0L.OBJ +
..\OBJ\TEST.OBJ      +
..\OBJ\OS_CPU_A.OBJ   +
..\OBJ\OS_CPU_C.OBJ   +
..\OBJ\OS_DEBUG.OBJ   +
..\OBJ\devices.OBJ    + *objectfile ที่เพิ่มเติม
..\OBJ\COM86.OBJ      +
..\OBJ\uCOS_II.OBJ
..\OBJ\TEST,..\OBJ\TEST
c:\borlandc\LIB\EMU.LIB +
c:\borlandc\LIB\MATH.LIB +
c:\borlandc\LIB\CL.LIB
```

* สามารถเพิ่มเติมส่วนที่เราสร้างขึ้นเองมาได้ ด้วยวิธีข้างต้น

หลังจากนั้นก็ทำการเขียนแฟ้มข้อมูลที่สั่งให้การสร้างแฟ้มข้อมูลที่เรียกใช้งานขึ้นมา (ใช้แฟ้มข้อมูลนามสกุล .BAT)

```
MD ..\WORK
MD ..\OBJ
MD ..\LST
CD ..\WORK
COPY ..\TEST\TEST.MAK TEST.MAK
c:\borlandc\BIN\MAKE -f TEST.MAK
CD ..\TEST
```

ทั้งหมดที่กล่าวมาเป็นโครงสร้างและลักษณะการทำงานของโปรแกรมที่เกี่ยวข้องทั้งหมดซึ่งผู้อ่านสามารถนำความรู้นี้ไปปรับแต่งใช้งานกับ โปรแกรมของโครงการนี้ได้พอสมควรแล้ว สำหรับผู้ที่ต้องการศึกษาให้ลึกกว่านี้สามารถอ่านได้ที่ thesis ของข้าพเจ้าและสามารถสืบค้นเพิ่มเติมได้จากส่วนบรรณานุกรมของ thesis ของข้าพเจ้าได้